

Introduction

1 WHAT IS A NEURAL NETWORK?

Work on artificial neural networks, commonly referred to as “neural networks,” has been motivated right from its inception by the recognition that the human brain computes in an entirely different way from the conventional digital computer. The brain is a highly *complex, nonlinear, and parallel computer* (information-processing system). It has the capability to organize its structural constituents, known as *neurons*, so as to perform certain computations (e.g., pattern recognition, perception, and motor control) many times faster than the fastest digital computer in existence today. Consider, for example, human *vision*, which is an information-processing task. It is the function of the visual system to provide a *representation* of the environment around us and, more important, to supply the information we need to *interact* with the environment. To be specific, the brain routinely accomplishes perceptual recognition tasks (e.g., recognizing a familiar face embedded in an unfamiliar scene) in approximately 100–200 ms, whereas tasks of much lesser complexity take a great deal longer on a powerful computer.

For another example, consider the *sonar* of a bat. Sonar is an active echolocation system. In addition to providing information about how far away a target (e.g., a flying insect) is, bat sonar conveys information about the relative velocity of the target, the size of the target, the size of various features of the target, and the azimuth and elevation of the target. The complex neural computations needed to extract all this information from the target echo occur within a brain the size of a plum. Indeed, an echolocating bat can pursue and capture its target with a facility and success rate that would be the envy of a radar or sonar engineer.

How, then, does a human brain or the brain of a bat do it? At birth, a brain already has considerable structure and the ability to build up its own rules of behavior through what we usually refer to as “experience.” Indeed, experience is built up over time, with much of the development (i.e., hardwiring) of the human brain taking place during the first two years from birth, but the development continues well beyond that stage.

A “developing” nervous system is synonymous with a plastic brain: *Plasticity* permits the developing nervous system to *adapt* to its surrounding environment. Just as plasticity appears to be essential to the functioning of neurons as information-processing units in the human brain, so it is with neural networks made up of artificial neurons. In

2 Introduction

its most general form, a *neural network* is a machine that is designed to *model* the way in which the brain performs a particular task or function of interest; the network is usually implemented by using electronic components or is simulated in software on a digital computer. In this book, we focus on an important class of neural networks that perform useful computations through a process of *learning*. To achieve good performance, neural networks employ a massive interconnection of simple computing cells referred to as “neurons” or “processing units.” We may thus offer the following definition of a neural network viewed as an adaptive machine¹:

A neural network is a massively parallel distributed processor made up of simple processing units that has a natural propensity for storing experiential knowledge and making it available for use. It resembles the brain in two respects:

1. *Knowledge is acquired by the network from its environment through a learning process.*
2. *Interneuron connection strengths, known as synaptic weights, are used to store the acquired knowledge.*

The procedure used to perform the learning process is called a *learning algorithm*, the function of which is to modify the synaptic weights of the network in an orderly fashion to attain a desired design objective.

The modification of synaptic weights provides the traditional method for the design of neural networks. Such an approach is the closest to linear adaptive filter theory, which is already well established and successfully applied in many diverse fields (Widrow and Stearns, 1985; Haykin, 2002). However, it is also possible for a neural network to modify its own topology, which is motivated by the fact that neurons in the human brain can die and new synaptic connections can grow.

Benefits of Neural Networks

It is apparent that a neural network derives its computing power through, first, its massively parallel distributed structure and, second, its ability to learn and therefore generalize. *Generalization* refers to the neural network’s production of reasonable outputs for inputs not encountered during training (learning). These two information-processing capabilities make it possible for neural networks to find good approximate solutions to complex (large-scale) problems that are *intractable*. In practice, however, neural networks cannot provide the solution by working individually. Rather, they need to be integrated into a consistent system engineering approach. Specifically, a complex problem of interest is *decomposed* into a number of relatively simple tasks, and neural networks are assigned a subset of the tasks that *match* their inherent capabilities. It is important to recognize, however, that we have a long way to go (if ever) before we can build a computer architecture that mimics the human brain.

Neural networks offer the following useful properties and capabilities:

1. **Nonlinearity.** An artificial neuron can be linear or nonlinear. A neural network, made up of an interconnection of nonlinear neurons, is itself nonlinear. Moreover, the nonlinearity is of a special kind in the sense that it is *distributed* throughout the network. Nonlinearity is a highly important property, particularly if the underlying physical

mechanism responsible for generation of the input signal (e.g., speech signal) is inherently nonlinear.

2. *Input–Output Mapping.* A popular paradigm of learning, called *learning with a teacher*, or *supervised learning*, involves modification of the synaptic weights of a neural network by applying a set of labeled *training examples*, or *task examples*. Each example consists of a unique *input signal* and a corresponding *desired (target) response*. The network is presented with an example picked at random from the set, and the synaptic weights (free parameters) of the network are modified to minimize the difference between the desired response and the actual response of the network produced by the input signal in accordance with an appropriate statistical criterion. The training of the network is repeated for many examples in the set, until the network reaches a steady state where there are no further significant changes in the synaptic weights. The previously applied training examples may be reapplied during the training session, but in a different order. Thus the network learns from the examples by constructing an *input–output mapping* for the problem at hand. Such an approach brings to mind the study of *nonparametric statistical inference*, which is a branch of statistics dealing with model-free estimation, or, from a biological viewpoint, *tabula rasa* learning (Geman et al., 1992); the term “nonparametric” is used here to signify the fact that no prior assumptions are made on a statistical model for the input data. Consider, for example, a *pattern classification* task, where the requirement is to assign an input signal representing a physical object or event to one of several prespecified categories (classes). In a nonparametric approach to this problem, the requirement is to “estimate” arbitrary decision boundaries in the input signal space for the pattern-classification task using a set of examples, and to do so *without* invoking a probabilistic distribution model. A similar point of view is implicit in the supervised learning paradigm, which suggests a close analogy between the input–output mapping performed by a neural network and nonparametric statistical inference.

3. *Adaptivity.* Neural networks have a built-in capability to *adapt* their synaptic weights to changes in the surrounding environment. In particular, a neural network trained to operate in a specific environment can be easily *retrained* to deal with minor changes in the operating environmental conditions. Moreover, when it is operating in a *nonstationary* environment (i.e., one where statistics change with time), a neural network may be designed to change its synaptic weights in real time. The natural architecture of a neural network for pattern classification, signal processing, and control applications, coupled with the adaptive capability of the network, makes it a useful tool in adaptive pattern classification, adaptive signal processing, and adaptive control. As a general rule, it may be said that the more adaptive we make a system, all the time ensuring that the system remains stable, the more robust its performance will likely be when the system is required to operate in a nonstationary environment. It should be emphasized, however, that adaptivity does not always lead to robustness; indeed, it may do the very opposite. For example, an adaptive system with short-time constants may change rapidly and therefore tend to respond to spurious disturbances, causing a drastic degradation in system performance. To realize the full benefits of adaptivity, the principal time constants of the system should be long enough for the system to ignore spurious disturbances, and yet short enough to respond to meaningful changes in the

environment; the problem described here is referred to as the *stability–plasticity dilemma* (Grossberg, 1988).

4. *Evidential Response.* In the context of pattern classification, a neural network can be designed to provide information not only about which particular pattern to *select*, but also about the *confidence* in the decision made. This latter information may be used to reject ambiguous patterns, should they arise, and thereby improve the classification performance of the network.

5. *Contextual Information.* Knowledge is represented by the very structure and activation state of a neural network. Every neuron in the network is potentially affected by the global activity of all other neurons in the network. Consequently, contextual information is dealt with naturally by a neural network.

6. *Fault Tolerance.* A neural network, implemented in hardware form, has the potential to be inherently *fault tolerant*, or capable of robust computation, in the sense that its performance degrades gracefully under adverse operating conditions. For example, if a neuron or its connecting links are damaged, recall of a stored pattern is impaired in quality. However, due to the distributed nature of information stored in the network, the damage has to be extensive before the overall response of the network is degraded seriously. Thus, in principle, a neural network exhibits a graceful degradation in performance rather than catastrophic failure. There is some empirical evidence for robust computation, but usually it is uncontrolled. In order to be assured that the neural network is, in fact, fault tolerant, it may be necessary to take corrective measures in designing the algorithm used to train the network (Kerlirzin and Vallet, 1993).

7. *VLSI Implementability.* The massively parallel nature of a neural network makes it potentially fast for the computation of certain tasks. This same feature makes a neural network well suited for implementation using *very-large-scale-integrated* (VLSI) technology. One particular beneficial virtue of VLSI is that it provides a means of capturing truly complex behavior in a highly hierarchical fashion (Mead, 1989).

8. *Uniformity of Analysis and Design.* Basically, neural networks enjoy universality as information processors. We say this in the sense that the same notation is used in all domains involving the application of neural networks. This feature manifests itself in different ways:

- Neurons, in one form or another, represent an ingredient *common* to all neural networks.
- This commonality makes it possible to *share* theories and learning algorithms in different applications of neural networks.
- Modular networks can be built through a *seamless integration of modules*.

9. *Neurobiological Analogy.* The design of a neural network is motivated by analogy with the brain, which is living proof that fault-tolerant parallel processing is not only physically possible, but also fast and powerful. Neurobiologists look to (artificial) neural networks as a research tool for the interpretation of neurobiological phenomena. On the other hand, engineers look to neurobiology for new ideas to solve problems more complex than those based on conventional hardwired design

techniques. These two viewpoints are illustrated by the following two respective examples:

- In Anastasio (1993), linear system models of the *vestibulo-ocular reflex* (VOR) are compared to neural network models based on *recurrent networks*, which are described in Section 6 and discussed in detail in Chapter 15. The vestibulo-ocular reflex is part of the oculomotor system. The function of VOR is to maintain visual (i.e., retinal) image stability by making eye rotations that are opposite to head rotations. The VOR is mediated by premotor neurons in the vestibular nuclei that receive and process head rotation signals from vestibular sensory neurons and send the results to the eye muscle motor neurons. The VOR is well suited for modeling because its input (head rotation) and its output (eye rotation) can be precisely specified. It is also a relatively simple reflex, and the neurophysiological properties of its constituent neurons have been well described. Among the three neural types, the premotor neurons (reflex interneurons) in the vestibular nuclei are the most complex and therefore most interesting. The VOR has previously been modeled using lumped, linear system descriptors and control theory. These models were useful in explaining some of the overall properties of the VOR, but gave little insight into the properties of its constituent neurons. This situation has been greatly improved through neural network modeling. Recurrent network models of VOR (programmed using an algorithm called real-time recurrent learning, described in Chapter 15) can reproduce and help explain many of the static, dynamic, nonlinear, and distributed aspects of signal processing by the neurons that mediate the VOR, especially the vestibular nuclei neurons.
- The *retina*, more than any other part of the brain, is where we begin to put together the relationships between the outside world represented by a visual sense, its *physical image* projected onto an array of receptors, and the first *neural images*. The retina is a thin sheet of neural tissue that lines the posterior hemisphere of the eyeball. The retina's task is to convert an optical image into a neural image for transmission down the optic nerve to a multitude of centers for further analysis. This is a complex task, as evidenced by the synaptic organization of the retina. In all vertebrate retinas, the transformation from optical to neural image involves three stages (Sterling, 1990):
 - (i) photo transduction by a layer of receptor neurons;
 - (ii) transmission of the resulting signals (produced in response to light) by chemical synapses to a layer of bipolar cells;
 - (iii) transmission of these signals, also by chemical synapses, to output neurons that are called ganglion cells.

At both synaptic stages (i.e., from receptor to bipolar cells, and from bipolar to ganglion cells), there are specialized laterally connected neurons called *horizontal cells* and *amacrine cells*, respectively. The task of these neurons is to modify the transmission across the synaptic layers. There are also centrifugal elements called *inter-plexiform cells*; their task is to convey signals from the inner synaptic layer back to the outer one. Some researchers have built electronic chips that mimic the structure of the retina. These electronic chips are called *neuromorphic* integrated circuits, a term coined by Mead (1989). A neuromorphic imaging sensor

consists of an array of photoreceptors combined with analog circuitry at each picture element (pixel). It emulates the retina in that it can adapt locally to changes in brightness, detect edges, and detect motion. The neurobiological analogy, exemplified by neuromorphic integrated circuits, is useful in another important way: It provides a hope and belief, and to a certain extent an existence of proof, that physical understanding of neurobiological structures could have a productive influence on the art of electronics and VLSI technology for the implementation of neural networks.

With inspiration from neurobiology in mind, it seems appropriate that we take a brief look at the human brain and its structural levels of organization.²

2 THE HUMAN BRAIN

The human nervous system may be viewed as a three-stage system, as depicted in the block diagram of Fig. 1 (Arbib, 1987). Central to the system is the *brain*, represented by the *neural (nerve) net*, which continually receives information, perceives it, and makes appropriate decisions. Two sets of arrows are shown in the figure. Those pointing from left to right indicate the *forward* transmission of information-bearing signals through the system. The arrows pointing from right to left (shown in red) signify the presence of *feedback* in the system. The *receptors* convert stimuli from the human body or the external environment into electrical impulses that convey information to the neural net (brain). The *effectors* convert electrical impulses generated by the neural net into discernible responses as system outputs.

The struggle to understand the brain has been made easier because of the pioneering work of Ramón y Cajál (1911), who introduced the idea of *neurons* as structural constituents of the brain. Typically, neurons are five to six orders of magnitude slower than silicon logic gates; events in a silicon chip happen in the nanosecond range, whereas neural events happen in the millisecond range. However, the brain makes up for the relatively slow rate of operation of a neuron by having a truly staggering number of neurons (nerve cells) with massive interconnections between them. It is estimated that there are approximately 10 billion neurons in the human cortex, and 60 trillion synapses or connections (Shepherd and Koch, 1990). The net result is that the brain is an enormously efficient structure. Specifically, the *energetic efficiency* of the brain is approximately 10^{-16} joules (J) per operation per second, whereas the corresponding value for the best computers is orders of magnitude larger.

Synapses, or *nerve endings*, are elementary structural and functional units that mediate the interactions between neurons. The most common kind of synapse is a *chemical synapse*, which operates as follows: A presynaptic process liberates a *transmitter* substance that diffuses across the synaptic junction between neurons and then acts on a post-synaptic process. Thus a synapse converts a presynaptic electrical signal into a chemical

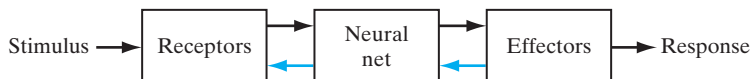


FIGURE 1 Block diagram representation of nervous system.

signal and then back into a postsynaptic electrical signal (Shepherd and Koch, 1990). In electrical terminology, such an element is said to be a *nonreciprocal two-port device*. In traditional descriptions of neural organization, it is assumed that a synapse is a simple connection that can impose *excitation* or *inhibition*, but not both on the receptive neuron.

Earlier we mentioned that plasticity permits the developing nervous system to adapt to its surrounding environment (Eggermont, 1990; Churchland and Sejnowski, 1992). In an adult brain, plasticity may be accounted for by two mechanisms: the creation of new synaptic connections between neurons, and the modification of existing synapses. *Axons*, the transmission lines, and *dendrites*, the receptive zones, constitute two types of cell filaments that are distinguished on morphological grounds; an axon has a smoother surface, fewer branches, and greater length, whereas a dendrite (so called because of its resemblance to a tree) has an irregular surface and more branches (Freeman, 1975). Neurons come in a wide variety of shapes and sizes in different parts of the brain. Figure 2 illustrates the shape of a *pyramidal cell*, which is one of the most common types of cortical neurons. Like many other types of neurons, it receives most of its inputs through dendritic spines; see the segment of dendrite in the insert in Fig. 2 for detail. The pyramidal cell can receive 10,000 or more synaptic contacts, and it can project onto thousands of target cells.

The majority of neurons encode their outputs as a series of brief voltage pulses. These pulses, commonly known as *action potentials*, or *spikes*,³ originate at or close to the cell body of neurons and then propagate across the individual neurons at constant velocity and amplitude. The reasons for the use of action potentials for communication among neurons are based on the physics of axons. The axon of a neuron is very long and thin and is characterized by high electrical resistance and very large capacitance. Both of these elements are distributed across the axon. The axon may therefore be modeled as resistance-capacitance (RC) transmission line, hence the common use of “cable equation” as the terminology for describing signal propagation along an axon. Analysis of this propagation mechanism reveals that when a voltage is applied at one end of the axon, it decays exponentially with distance, dropping to an insignificant level by the time it reaches the other end. The action potentials provide a way to circumvent this transmission problem (Anderson, 1995).

In the brain, there are both small-scale and large-scale anatomical organizations, and different functions take place at lower and higher levels. Figure 3 shows a hierarchy of interwoven levels of organization that has emerged from the extensive work done on the analysis of local regions in the brain (Shepherd and Koch, 1990; Churchland and Sejnowski, 1992). The *synapses* represent the most fundamental level, depending on molecules and ions for their action. At the next levels, we have neural microcircuits, dendritic trees, and then neurons. A *neural microcircuit* refers to an assembly of synapses organized into patterns of connectivity to produce a functional operation of interest. A neural microcircuit may be likened to a silicon chip made up of an assembly of transistors. The smallest size of microcircuits is measured in micrometers (μm), and their fastest speed of operation is measured in milliseconds. The neural microcircuits are grouped to form *dendritic subunits* within the *dendritic trees* of individual neurons. The whole *neuron*, about 100 μm in size, contains several dendritic subunits. At the next level of complexity, we have *local circuits* (about 1 mm in size) made up of neurons with similar or different properties; these neural

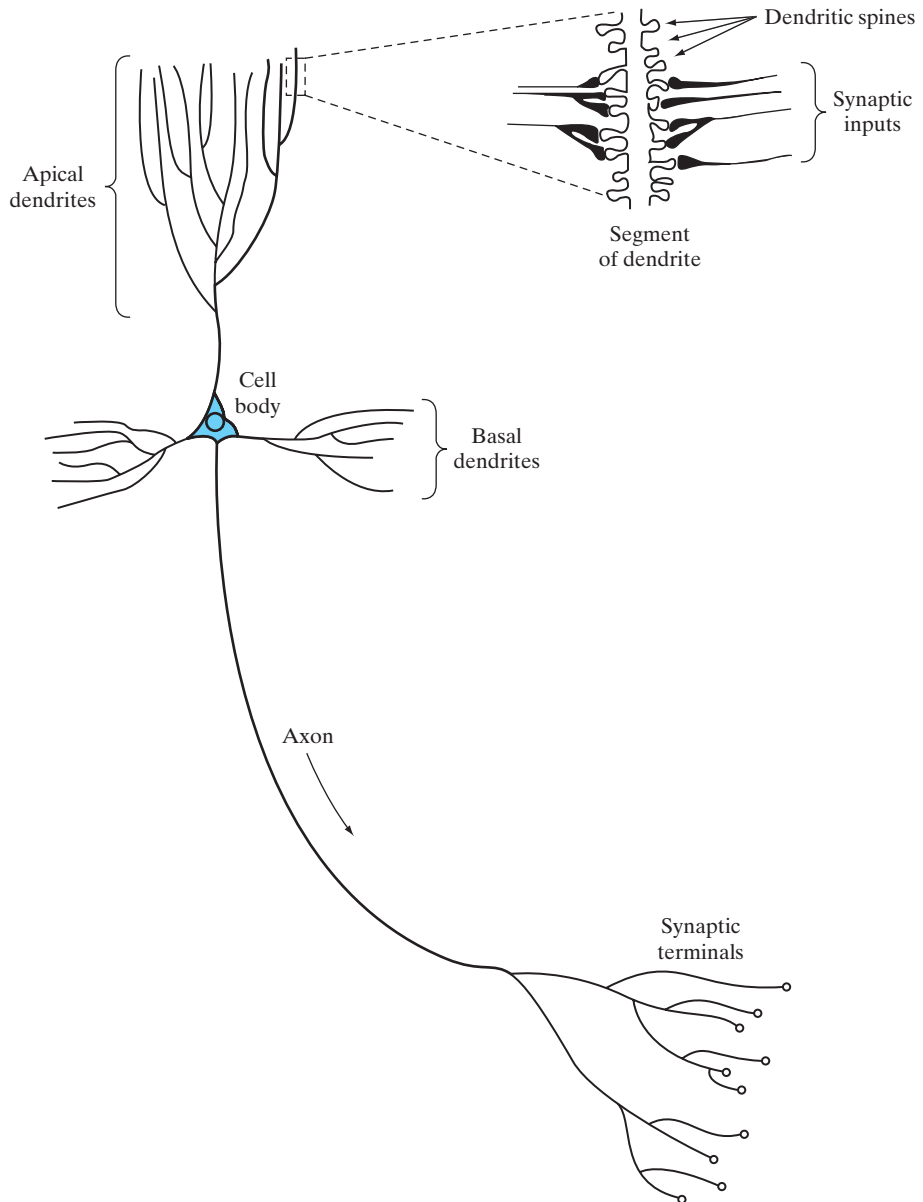


FIGURE 2 The pyramidal cell.

assemblies perform operations characteristic of a localized region in the brain. They are followed by *interregional circuits* made up of pathways, columns, and topographic maps, which involve multiple regions located in different parts of the brain.

Topographic maps are organized to respond to incoming sensory information. These maps are often arranged in sheets, as in the *superior colliculus*, where the visual,

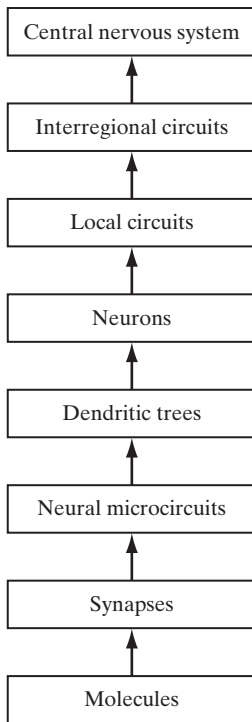


FIGURE 3 Structural organization of levels in the brain.

auditory, and somatosensory maps are stacked in adjacent layers in such a way that stimuli from corresponding points in space lie above or below each other. Figure 4 presents a cytoarchitectural map of the cerebral cortex as worked out by Brodmann (Brodal, 1981). This figure shows clearly that different sensory inputs (motor, somatosensory, visual, auditory, etc.) are mapped onto corresponding areas of the cerebral cortex in an orderly fashion. At the final level of complexity, the topographic maps and other interregional circuits mediate specific types of behavior in the *central nervous system*.

It is important to recognize that the structural levels of organization described herein are a unique characteristic of the brain. They are nowhere to be found in a digital computer, and we are nowhere close to re-creating them with artificial neural networks. Nevertheless, we are inching our way toward a hierarchy of computational levels similar to that described in Fig. 3. The artificial neurons we use to build our neural networks are truly primitive in comparison with those found in the brain. The neural networks we are presently able to design are just as primitive compared with the local circuits and the interregional circuits in the brain. What is really satisfying, however, is the remarkable progress that we have made on so many fronts. With neurobiological analogy as the source of inspiration, and the wealth of theoretical and computational tools that we are bringing together, it is certain that our understanding of artificial neural networks and their applications will continue to grow in depth as well as breadth, year after year.

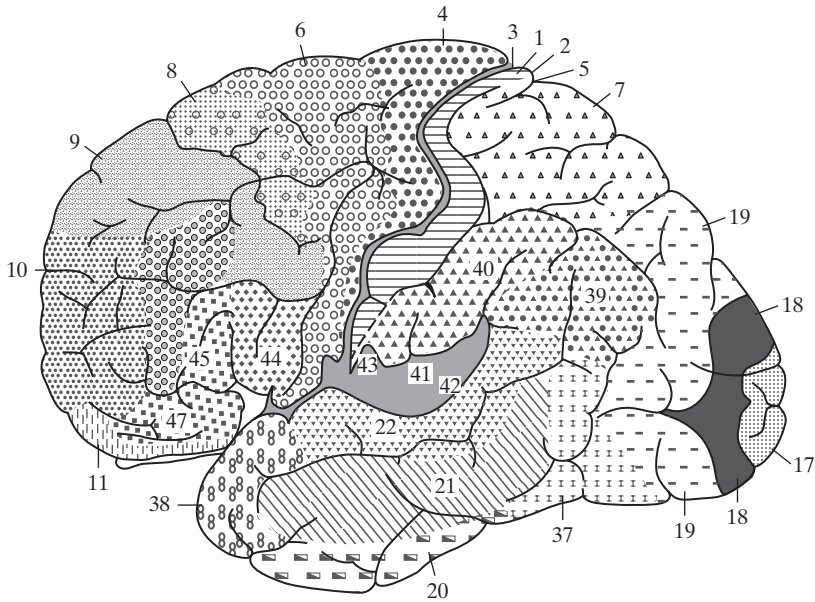


FIGURE 4 Cytoarchitectural map of the cerebral cortex. The different areas are identified by the thickness of their layers and types of cells within them. Some of the key sensory areas are as follows: Motor cortex: motor strip, area 4; premotor area, area 6; frontal eye fields, area 8. Somatosensory cortex: areas 3, 1, and 2. Visual cortex: areas 17, 18, and 19. Auditory cortex: areas 41 and 42. (From A. Brodal, 1981; with permission of Oxford University Press.)

3 MODELS OF A NEURON

A *neuron* is an information-processing unit that is fundamental to the operation of a neural network. The block diagram of Fig. 5 shows the *model* of a neuron, which forms the basis for designing a large family of neural networks studied in later chapters. Here, we identify three basic elements of the neural model:

1. A set of *synapses*, or *connecting links*, each of which is characterized by a *weight* or *strength* of its own. Specifically, a signal x_j at the input of synapse j connected to neuron k is multiplied by the synaptic weight w_{kj} . It is important to make a note of the manner in which the subscripts of the synaptic weight w_{kj} are written. The first subscript in w_{kj} refers to the neuron in question, and the second subscript refers to the input end of the synapse to which the weight refers. Unlike the weight of a synapse in the brain, the synaptic weight of an artificial neuron may lie in a range that includes negative as well as positive values.
2. An *adder* for summing the input signals, weighted by the respective synaptic strengths of the neuron; the operations described here constitute a *linear combiner*.
3. An *activation function* for limiting the amplitude of the output of a neuron. The activation function is also referred to as a *squashing function*, in that it squashes (limits) the permissible amplitude range of the output signal to some finite value.

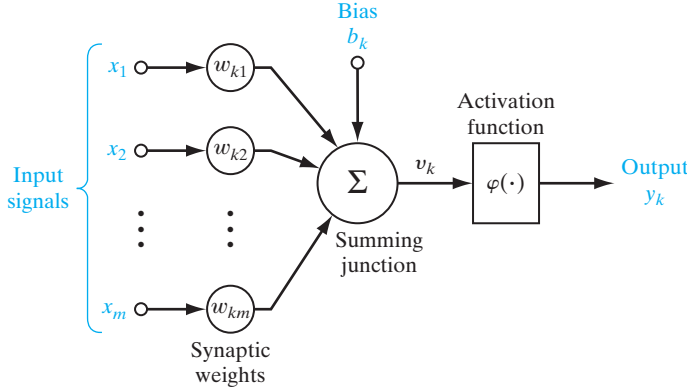


FIGURE 5 Nonlinear model of a neuron, labeled k .

Typically, the normalized amplitude range of the output of a neuron is written as the closed unit interval $[0,1]$, or, alternatively, $[-1,1]$.

The neural model of Fig. 5 also includes an externally applied *bias*, denoted by b_k . The bias b_k has the effect of increasing or lowering the net input of the activation function, depending on whether it is positive or negative, respectively.

In mathematical terms, we may describe the neuron k depicted in Fig. 5 by writing the pair of equations:

$$u_k = \sum_{j=1}^m w_{kj} x_j \quad (1)$$

and

$$y_k = \varphi(u_k + b_k) \quad (2)$$

where $x_1, x_2, \dots, x_m$ are the input signals; $w_{k1}, w_{k2}, \dots, w_{km}$ are the respective synaptic weights of neuron k ; u_k (not shown in Fig. 5) is the *linear combiner output* due to the input signals; b_k is the bias; $\varphi(\cdot)$ is the *activation function*; and y_k is the output signal of the neuron. The use of bias b_k has the effect of applying an *affine transformation* to the output u_k of the linear combiner in the model of Fig. 5, as shown by

$$v_k = u_k + b_k \quad (3)$$

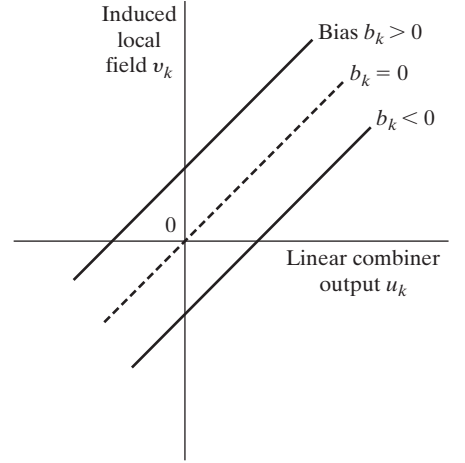
In particular, depending on whether the bias b_k is positive or negative, the relationship between the *induced local field*, or *activation potential*, v_k of neuron k and the linear combiner output u_k is modified in the manner illustrated in Fig. 6; hereafter, these two terms are used interchangeably. Note that as a result of this affine transformation, the graph of v_k versus u_k no longer passes through the origin.

The bias b_k is an external parameter of neuron k . We may account for its presence as in Eq. (2). Equivalently, we may formulate the combination of Eqs. (1) to (3) as follows:

$$v_k = \sum_{j=0}^m w_{kj} x_j \quad (4)$$

12 Introduction

FIGURE 6 Affine transformation produced by the presence of a bias; note that $v_k = b_k$ at $u_k = 0$.



and

$$y_k = \varphi(v_k) \quad (5)$$

In Eq. (4), we have added a new synapse. Its input is

$$x_0 = +1 \quad (6)$$

and its weight is

$$w_{k0} = b_k \quad (7)$$

We may therefore reformulate the model of neuron k as shown in Fig. 7. In this figure, the effect of the bias is accounted for by doing two things: (1) adding a new input signal fixed at $+1$, and (2) adding a new synaptic weight equal to the bias b_k . Although the models of Figs. 5 and 7 are different in appearance, they are mathematically equivalent.

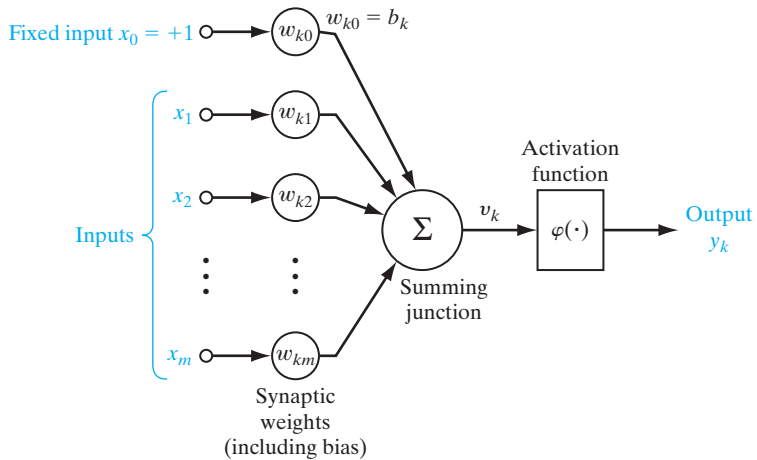


FIGURE 7 Another nonlinear model of a neuron; w_{k0} accounts for the bias b_k .

Types of Activation Function

The activation function, denoted by $\varphi(v)$, defines the output of a neuron in terms of the induced local field v . In what follows, we identify two basic types of activation functions:

1. **Threshold Function.** For this type of activation function, described in Fig. 8a, we have

$$\varphi(v) = \begin{cases} 1 & \text{if } v \geq 0 \\ 0 & \text{if } v < 0 \end{cases} \quad (8)$$

In engineering, this form of a threshold function is commonly referred to as a *Heaviside function*. Correspondingly, the output of neuron k employing such a threshold function is expressed as

$$y_k = \begin{cases} 1 & \text{if } v_k \geq 0 \\ 0 & \text{if } v_k < 0 \end{cases} \quad (9)$$

where v_k is the induced local field of the neuron; that is,

$$v_k = \sum_{j=1}^m w_{kj}x_j + b_k \quad (10)$$

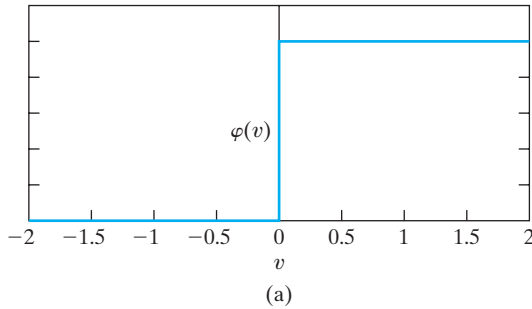
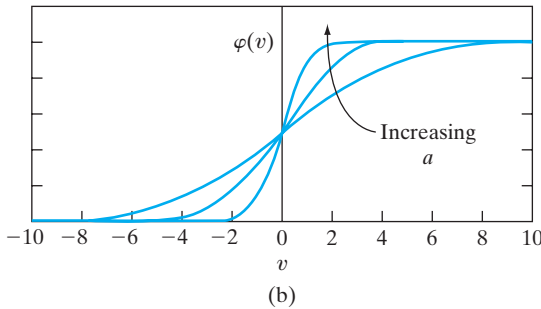


FIGURE 8 (a) Threshold function. (b) Sigmoid function for varying slope parameter a .



In neural computation, such a neuron is referred to as the *McCulloch–Pitts model*, in recognition of the pioneering work done by McCulloch and Pitts (1943). In this model, the output of a neuron takes on the value of 1 if the induced local field of that neuron is nonnegative, and 0 otherwise. This statement describes the *all-or-none property* of the McCulloch–Pitts model.

2. Sigmoid Function.⁴ The sigmoid function, whose graph is “S”-shaped, is by far the most common form of activation function used in the construction of neural networks. It is defined as a strictly increasing function that exhibits a graceful balance between linear and nonlinear behavior. An example of the sigmoid function is the *logistic function*,⁵ defined by

$$\varphi(v) = \frac{1}{1 + \exp(-av)} \quad (11)$$

where a is the *slope parameter* of the sigmoid function. By varying the parameter a , we obtain sigmoid functions of different slopes, as illustrated in Fig. 8b. In fact, the slope at the origin equals $a/4$. In the limit, as the slope parameter approaches infinity, the sigmoid function becomes simply a threshold function. Whereas a threshold function assumes the value of 0 or 1, a sigmoid function assumes a continuous range of values from 0 to 1. Note also that the sigmoid function is differentiable, whereas the threshold function is not. (Differentiability is an important feature of neural network theory, as described in Chapter 4).

The activation functions defined in Eqs. (8) and (11) range from 0 to +1. It is sometimes desirable to have the activation function range from -1 to $+1$, in which case, the activation function is an odd function of the induced local field. Specifically, the threshold function of Eq. (8) is now defined as

$$\varphi(v) = \begin{cases} 1 & \text{if } v > 0 \\ 0 & \text{if } v = 0 \\ -1 & \text{if } v < 0 \end{cases} \quad (12)$$

which is commonly referred to as the *signum function*. For the corresponding form of a sigmoid function, we may use the *hyperbolic tangent function*, defined by

$$\varphi(v) = \tanh(v) \quad (13)$$

Allowing an activation function of the sigmoid type to assume negative values as prescribed by Eq. (13) may yield practical benefits over the logistic function of Eq. (11).

Stochastic Model of a Neuron

The neural model described in Fig. 7 is deterministic in that its input–output behavior is precisely defined for all inputs. For some applications of neural networks, it is desirable to base the analysis on a stochastic neural model. In an analytically tractable approach, the activation function of the McCulloch–Pitts model is given a probabilistic interpretation. Specifically, a neuron is permitted to reside in only one of two states: $+1$

or -1 , say. The decision for a neuron to *fire* (i.e., switch its state from “off” to “on”) is probabilistic. Let x denote the state of the neuron and $P(v)$ denote the *probability* of firing, where v is the induced local field of the neuron. We may then write

$$x = \begin{cases} +1 & \text{with probability } P(v) \\ -1 & \text{with probability } 1 - P(v) \end{cases} \quad (14)$$

A standard choice for $P(v)$ is the sigmoid-shaped function

$$P(v) = \frac{1}{1 + \exp(-v/T)} \quad (15)$$

where T is a *pseudotemperature* used to control the noise level and therefore the uncertainty in firing (Little, 1974). It is important to realize, however, that T is *not* the physical temperature of a neural network, be it a biological or an artificial neural network. Rather, as already stated, we should think of T merely as a parameter that controls the thermal fluctuations representing the effects of synaptic noise. Note that when $T \rightarrow 0$, the stochastic neuron described by Eqs. (14) and (15) reduces to a noiseless (i.e., deterministic) form, namely, the McCulloch–Pitts model.

4 NEURAL NETWORKS VIEWED AS DIRECTED GRAPHS

The *block diagram* of Fig. 5 or that of Fig. 7 provides a functional description of the various elements that constitute the model of an artificial neuron. We may simplify the appearance of the model by using the idea of signal-flow graphs without sacrificing any of the functional details of the model. Signal-flow graphs, with a well-defined set of rules, were originally developed by Mason (1953, 1956) for linear networks. The presence of nonlinearity in the model of a neuron limits the scope of their application to neural networks. Nevertheless, signal-flow graphs do provide a neat method for the portrayal of the flow of signals in a neural network, which we pursue in this section.

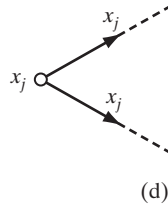
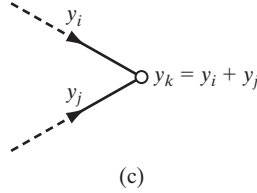
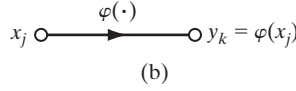
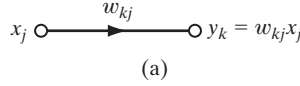
A *signal-flow graph* is a network of directed *links* (*branches*) that are interconnected at certain points called *nodes*. A typical node j has an associated *node signal* x_j . A typical directed link originates at node j and terminates on node k ; it has an associated *transfer function*, or *transmittance*, that specifies the manner in which the signal y_k at node k depends on the signal x_j at node j . The flow of signals in the various parts of the graph is dictated by three basic rules:

Rule 1. A signal flows along a link only in the direction defined by the arrow on the link.

Two different types of links may be distinguished:

- *Synaptic links*, whose behavior is governed by a *linear* input–output relation. Specifically, the node signal x_j is multiplied by the synaptic weight w_{kj} to produce the node signal y_k , as illustrated in Fig. 9a.
- *Activation links*, whose behavior is governed in general by a *nonlinear* input–output relation. This form of relationship is illustrated in Fig. 9b, where $\varphi(\cdot)$ is the nonlinear activation function.

FIGURE 9 Illustrating basic rules for the construction of signal-flow graphs.



Rule 2. A node signal equals the algebraic sum of all signals entering the pertinent node via the incoming links.

This second rule is illustrated in Fig. 9c for the case of *synaptic convergence*, or *fan-in*.

Rule 3. The signal at a node is transmitted to each outgoing link originating from that node, with the transmission being entirely independent of the transfer functions of the outgoing links.

This third rule is illustrated in Fig. 9d for the case of *synaptic divergence*, or *fan-out*.

For example, using these rules, we may construct the signal-flow graph of Fig. 10 as the model of a neuron, corresponding to the block diagram of Fig. 7. The representation shown in Fig. 10 is clearly simpler in appearance than that of Fig. 7, yet it contains all the functional details depicted in the latter diagram. Note that in both figures, the input $x_0 = +1$ and the associated synaptic weight $w_{k0} = b_k$, where b_k is the bias applied to neuron k .

Indeed, based on the signal-flow graph of Fig. 10 as the model of a neuron, we may now offer the following mathematical definition of a neural network:

A neural network is a directed graph consisting of nodes with interconnecting synaptic and activation links and is characterized by four properties:

1. *Each neuron is represented by a set of linear synaptic links, an externally applied bias, and a possibly nonlinear activation link. The bias is represented by a synaptic link connected to an input fixed at +1.*
2. *The synaptic links of a neuron weight their respective input signals.*

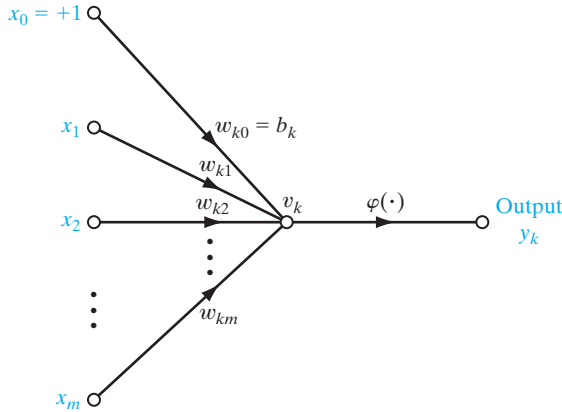


FIGURE 10 Signal-flow graph of a neuron.

3. The weighted sum of the input signals defines the induced local field of the neuron in question.
4. The activation link squashes the induced local field of the neuron to produce an output.

A directed graph, defined in this manner is *complete* in the sense that it describes not only the signal flow from neuron to neuron, but also the signal flow inside each neuron. When, however, the focus of attention is restricted to signal flow from neuron to neuron, we may use a reduced form of this graph by omitting the details of signal flow inside the individual neurons. Such a directed graph is said to be *partially complete*. It is characterized as follows:

1. Source nodes supply input signals to the graph.
2. Each neuron is represented by a single node called a *computation node*.
3. The *communication links* interconnecting the source and computation nodes of the graph carry no weight; they merely provide directions of signal flow in the graph.

A partially complete directed graph defined in this way is referred to as an *architectural graph*, describing the layout of the neural network. It is illustrated in Fig. 11 for the simple case of a single neuron with m source nodes and a single node fixed at $+1$ for the bias. Note that the computation node representing the neuron is shown shaded, and the source node is shown as a small square. This convention is followed throughout the book. More elaborate examples of architectural layouts are presented later in Section 6.

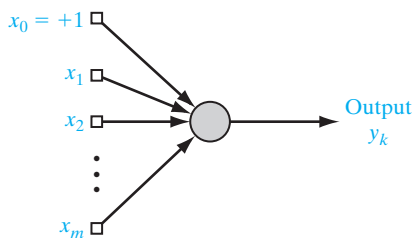


FIGURE 11 Architectural graph of a neuron.

To sum up, we have three graphical representations of a neural network:

- block diagram, providing a functional description of the network;
- architectural graph, describing the network layout;
- signal-flow graph, providing a complete description of signal flow in the network.

5 FEEDBACK

Feedback is said to exist in a dynamic system whenever the output of an element in the system influences in part the input applied to that particular element, thereby giving rise to one or more closed paths for the transmission of signals around the system. Indeed, feedback occurs in almost every part of the nervous system of every animal (Freeman, 1975). Moreover, it plays a major role in the study of a special class of neural networks known as *recurrent networks*. Figure 12 shows the signal-flow graph of a *single-loop feedback system*, where the input signal $x_j(n)$, internal signal $x'_j(n)$, and output signal $y_k(n)$ are functions of the discrete-time variable n . The system is assumed to be *linear*, consisting of a forward path and a feedback path that are characterized by the “operators” **A** and **B**, respectively. In particular, the output of the forward channel determines in part its own output through the feedback channel. From Fig. 12, we readily note the input–output relationships

$$y_k(n) = \mathbf{A}[x'_j(n)] \quad (16)$$

and

$$x'_j(n) = x_j(n) + \mathbf{B}[y_k(n)] \quad (17)$$

where the square brackets are included to emphasize that **A** and **B** act as *operators*. Eliminating $x'_j(n)$ between Eqs. (16) and (17), we get

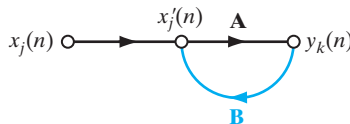
$$y_k(n) = \frac{\mathbf{A}}{1 - \mathbf{AB}}[x_j(n)] \quad (18)$$

We refer to $\mathbf{A}/(1 - \mathbf{AB})$ as the *closed-loop operator* of the system, and to $\mathbf{AB}$ as the *open-loop operator*. In general, the open-loop operator is noncommutative in that $\mathbf{BA} \neq \mathbf{AB}$.

Consider, for example, the single-loop feedback system shown in Fig. 13a, for which **A** is a fixed weight w and **B** is a *unit-delay operator* z^{-1} , whose output is delayed with respect to the input by one time unit. We may then express the closed-loop operator of the system as

$$\begin{aligned} \frac{\mathbf{A}}{1 - \mathbf{AB}} &= \frac{w}{1 - wz^{-1}} \\ &= w(1 - wz^{-1})^{-1} \end{aligned}$$

FIGURE 12 Signal-flow graph of a single-loop feedback system.



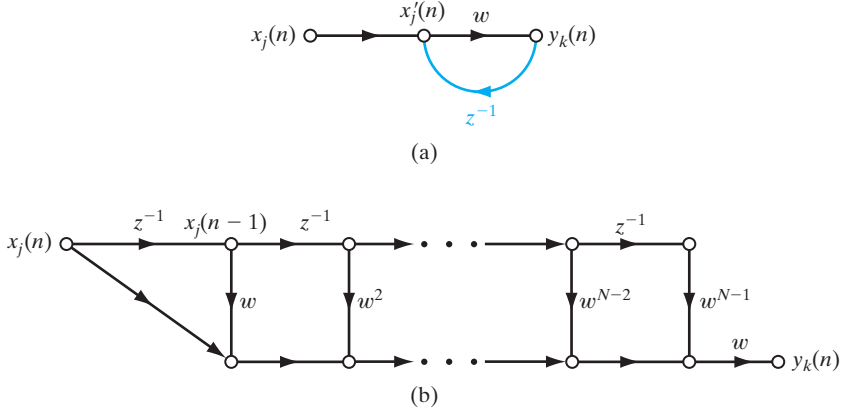


FIGURE 13 (a) Signal-flow graph of a first-order, infinite-duration impulse response (IIR) filter. (b) Feedforward approximation of part (a) of the figure, obtained by truncating Eq. (20).

Using the binomial expansion for $(1 - wz^{-1})^{-1}$, we may rewrite the closed-loop operator of the system as

$$\frac{\mathbf{A}}{1 - \mathbf{A}\mathbf{B}} = w \sum_{l=0}^{\infty} w^l z^{-l} \quad (19)$$

Hence, substituting Eq. (19) into (18), we get

$$y_k(n) = w \sum_{l=0}^{\infty} w^l z^{-l} [x_j(n)] \quad (20)$$

where again we have included square brackets to emphasize the fact that z^{-1} is an operator. In particular, from the definition of z^{-1} , we have

$$z^{-l} [x_j(n)] = x_j(n - l) \quad (21)$$

where $x_j(n - l)$ is a sample of the input signal delayed by l time units. Accordingly, we may express the output signal $y_k(n)$ as an infinite weighted summation of present and past samples of the input signal $x_j(n)$, as shown by

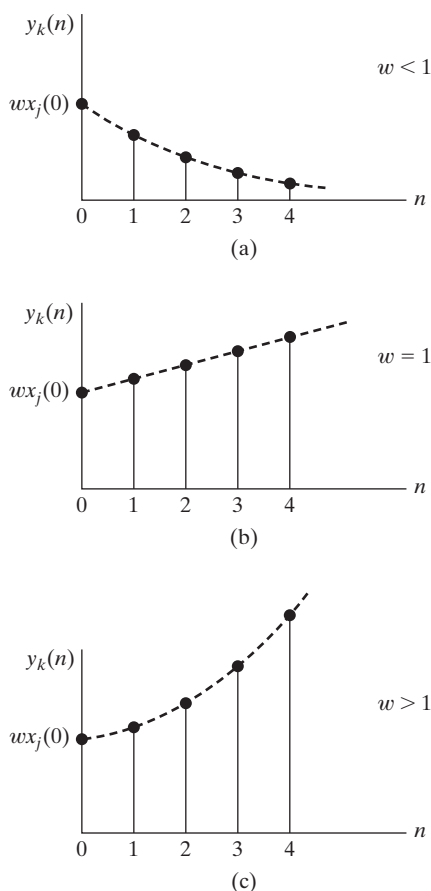
$$y_k(n) = \sum_{l=0}^{\infty} w^{l+1} x_j(n - l) \quad (22)$$

We now see clearly that the dynamic behavior of a feedback system represented by the signal-flow graph of Fig. 13 is controlled by the weight w . In particular, we may distinguish two specific cases:

1. $|w| < 1$, for which the output signal $y_k(n)$ is exponentially *convergent*; that is, the system is *stable*. This case is illustrated in Fig. 14a for a positive w .
2. $|w| \geq 1$, for which the output signal $y_k(n)$ is *divergent*; that is, the system is *unstable*. If $|w| = 1$ the divergence is linear, as in Fig. 14b, and if $|w| > 1$ the divergence is exponential, as in Fig. 14c.

FIGURE 14 Time response of Fig. 13 for three different values of feedforward weight w .

- (a) Stable.
 (b) Linear divergence.
 (c) Exponential divergence.



The issue of stability features prominently in the study of closed-loop feedback systems.

The case of $|w| < 1$ corresponds to a system with *infinite memory* in the sense that the output of the system depends on samples of the input extending into the infinite past. Moreover, the memory is *fading* in that the influence of a past sample is reduced exponentially with time n . Suppose that, for some power N , $|w|^N$ is small enough relative to unity such that w^N is negligible for all practical purposes. In such a situation, we may approximate the output y_k by the finite sum

$$\begin{aligned}
 y_k(n) &\approx \sum_{l=0}^{N-1} w^{l+1} x_j(n-l) \\
 &= wx_j(n) + w^2 x_j(n-1) + w^3 x_j(n-2) + \dots + w^N x_j(n-N+1)
 \end{aligned}$$

In a corresponding way, we may use the feedforward signal-flow graph of Fig. 13b as the approximation for the feedback signal-flow graph of Fig. 13a. In making this approximation, we speak of the “unfolding” of a feedback system. Note, however, that the unfolding operation is of practical value only when the feedback system is stable.

The analysis of the dynamic behavior of neural networks involving the application of feedback is unfortunately complicated by the fact that the processing units used for the construction of the network are usually *nonlinear*. Further consideration of this important issue is deferred to the latter part of the book.

6 NETWORK ARCHITECTURES

The manner in which the neurons of a neural network are structured is intimately linked with the learning algorithm used to train the network. We may therefore speak of learning algorithms (rules) used in the design of neural networks as being *structured*. The classification of learning algorithms is considered in Section 8. In this section, we focus attention on network architectures (structures).

In general, we may identify three fundamentally different classes of network architectures:

(i) Single-Layer Feedforward Networks

In a *layered* neural network, the neurons are organized in the form of layers. In the simplest form of a layered network, we have an *input layer* of source nodes that projects directly onto an *output layer* of neurons (computation nodes), but not vice versa. In other words, this network is strictly of a *feedforward* type. It is illustrated in Fig. 15 for the case of four nodes in both the input and output layers. Such a network is called a *single-layer network*, with the designation “single-layer” referring to the output layer of computation nodes (neurons). We do not count the input layer of source nodes because no computation is performed there.

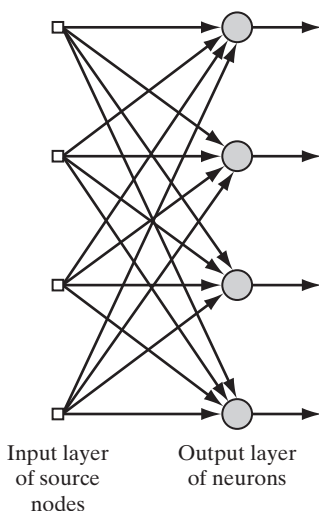


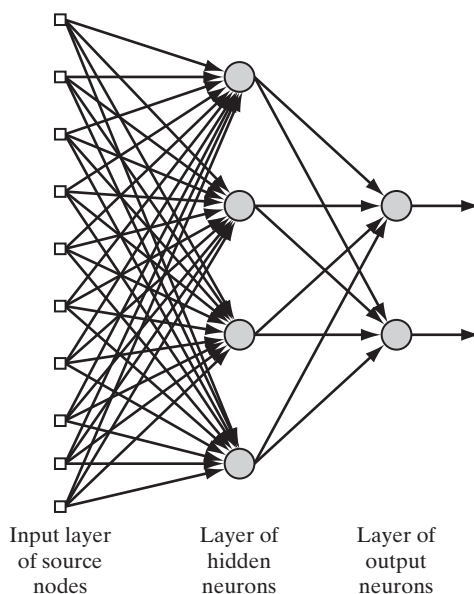
FIGURE 15 Feedforward network with a single layer of neurons.

(ii) Multilayer Feedforward Networks

The second class of a feedforward neural network distinguishes itself by the presence of one or more *hidden layers*, whose computation nodes are correspondingly called *hidden neurons* or *hidden units*; the term “hidden” refers to the fact that this part of the neural network is not seen directly from either the input or output of the network. The function of hidden neurons is to intervene between the external input and the network output in some useful manner. By adding one or more hidden layers, the network is enabled to extract higher-order statistics from its input. In a rather loose sense, the network acquires a *global* perspective despite its local connectivity, due to the extra set of synaptic connections and the extra dimension of neural interactions (Churchland and Sejnowski, 1992).

The source nodes in the input layer of the network supply respective elements of the activation pattern (input vector), which constitute the input signals applied to the neurons (computation nodes) in the second layer (i.e., the first hidden layer). The output signals of the second layer are used as inputs to the third layer, and so on for the rest of the network. Typically, the neurons in each layer of the network have as their inputs the output signals of the preceding layer only. The set of output signals of the neurons in the output (final) layer of the network constitutes the overall response of the network to the activation pattern supplied by the source nodes in the input (first) layer. The architectural graph in Fig. 16 illustrates the layout of a multilayer feedforward neural network for the case of a single hidden layer. For the sake of brevity, the network in Fig. 16 is referred to as a 10–4–2 network because it has 10 source nodes, 4 hidden neurons, and 2 output neurons. As another example, a feedforward network with m source nodes, h_1 neurons in the first hidden layer, h_2 neurons in the second hidden layer, and q neurons in the output layer is referred to as an m – h_1 – h_2 – q network.

FIGURE 16 Fully connected feedforward network with one hidden layer and one output layer.



The neural network in Fig. 16 is said to be *fully connected* in the sense that every node in each layer of the network is connected to every other node in the adjacent forward layer. If, however, some of the communication links (synaptic connections) are missing from the network, we say that the network is *partially connected*.

(iii) Recurrent Networks

A *recurrent neural network* distinguishes itself from a feedforward neural network in that it has at least one *feedback* loop. For example, a recurrent network may consist of a single layer of neurons with each neuron feeding its output signal back to the inputs of all the other neurons, as illustrated in the architectural graph in Fig. 17. In the structure depicted in this figure, there are *no* self-feedback loops in the network; self-feedback refers to a situation where the output of a neuron is fed back into its own input. The recurrent network illustrated in Fig. 17 also has *no* hidden neurons.

In Fig. 18 we illustrate another class of recurrent networks with hidden neurons. The feedback connections shown in Fig. 18 originate from the hidden neurons as well as from the output neurons.

The presence of feedback loops, be it in the recurrent structure of Fig. 17 or in that of Fig. 18, has a profound impact on the learning capability of the network and on its performance. Moreover, the feedback loops involve the use of particular branches composed of unit-time delay elements (denoted by z^{-1}), which result in a nonlinear dynamic behavior, assuming that the neural network contains nonlinear units.

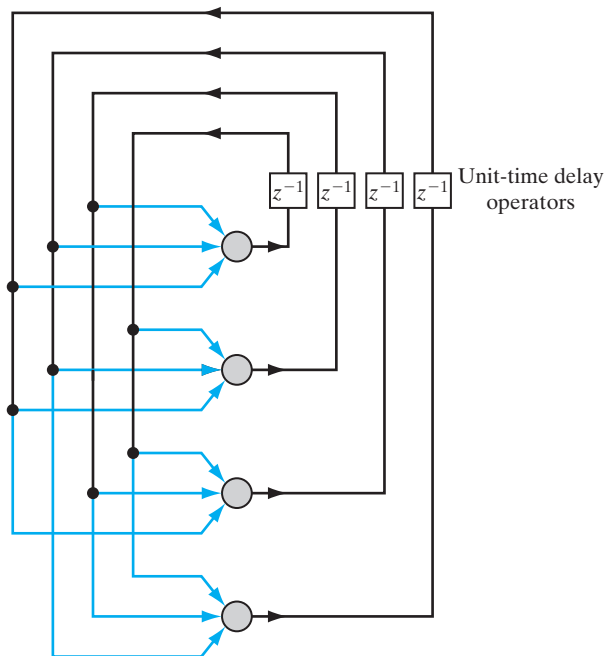


FIGURE 17 Recurrent network with no self-feedback loops and no hidden neurons.

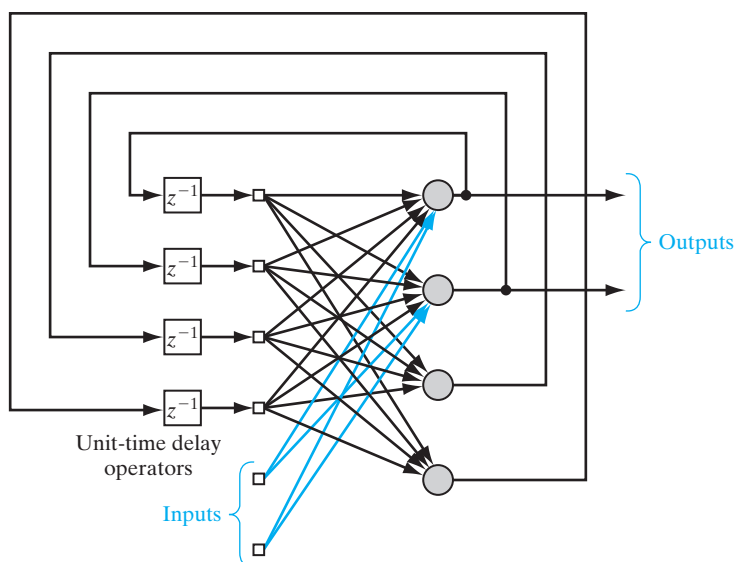


FIGURE 18 Recurrent network with hidden neurons.

7 KNOWLEDGE REPRESENTATION

In Section 1, we used the term “knowledge” in the definition of a neural network without an explicit description of what we mean by it. We now take care of this matter by offering the following generic definition (Fischler and Firschein, 1987):

Knowledge refers to stored information or models used by a person or machine to interpret, predict, and appropriately respond to the outside world.

The primary characteristics of *knowledge representation* are twofold: (1) what information is actually made explicit; and (2) how the information is physically encoded for subsequent use. By the very nature of it, therefore, knowledge representation is goal directed. In real-world applications of “intelligent” machines, it can be said that a good solution depends on a good representation of knowledge (Woods, 1986). So it is with neural networks. Typically, however, we find that the possible forms of representation from the inputs to internal network parameters are highly diverse, which tends to make the development of a satisfactory solution by means of a neural network a real design challenge.

A major task for a neural network is to learn a model of the world (environment) in which it is embedded, and to maintain the model sufficiently consistently with the real world so as to achieve the specified goals of the application of interest. Knowledge of the world consists of two kinds of information:

1. The known world state, represented by facts about what is and what has been known; this form of knowledge is referred to as *prior information*.
2. Observations (measurements) of the world, obtained by means of sensors designed to probe the environment, in which the neural network is supposed to operate.

Rosenblatt's Perceptron

ORGANIZATION OF THE CHAPTER

The perceptron occupies a special place in the historical development of neural networks: It was the first algorithmically described neural network. Its invention by Rosenblatt, a psychologist, inspired engineers, physicists, and mathematicians alike to devote their research effort to different aspects of neural networks in the 1960s and the 1970s. Moreover, it is truly remarkable to find that the perceptron (in its basic form as described in this chapter) is as valid today as it was in 1958 when Rosenblatt's paper on the perceptron was first published.

The chapter is organized as follows:

1. Section 1.1 expands on the formative years of neural networks, going back to the pioneering work of McCulloch and Pitts in 1943.
2. Section 1.2 describes Rosenblatt's perceptron in its most basic form. It is followed by Section 1.3 on the perceptron convergence theorem. This theorem proves convergence of the perceptron as a linearly separable pattern classifier in a finite number time-steps.
3. Section 1.4 establishes the relationship between the perceptron and the Bayes classifier for a Gaussian environment.
4. The experiment presented in Section 1.5 demonstrates the pattern-classification capability of the perceptron.
5. Section 1.6 generalizes the discussion by introducing the perceptron cost function, paving the way for deriving the batch version of the perceptron convergence algorithm.

Section 1.7 provides a summary and discussion that conclude the chapter.

1.1 INTRODUCTION

In the formative years of neural networks (1943–1958), several researchers stand out for their pioneering contributions:

- McCulloch and Pitts (1943) for introducing the idea of neural networks as computing machines.

- Hebb (1949) for postulating the first rule for self-organized learning.
- Rosenblatt (1958) for proposing the perceptron as the first model for learning with a teacher (i.e., supervised learning).

The idea of Hebbian learning will be discussed at some length in Chapter 8. In this chapter, we discuss Rosenblatt's *perceptron*.

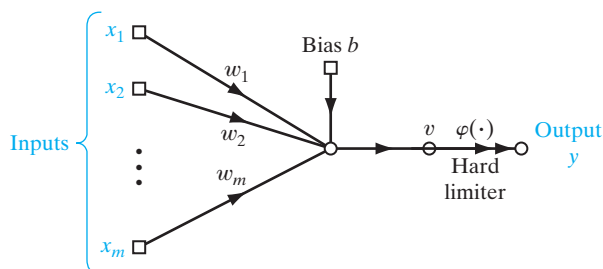
The perceptron is the simplest form of a neural network used for the classification of patterns said to be *linearly separable* (i.e., patterns that lie on opposite sides of a hyperplane). Basically, it consists of a single neuron with adjustable synaptic weights and bias. The algorithm used to adjust the free parameters of this neural network first appeared in a learning procedure developed by Rosenblatt (1958, 1962) for his perceptron brain model.¹ Indeed, Rosenblatt proved that if the patterns (vectors) used to train the perceptron are drawn from two linearly separable classes, then the perceptron algorithm converges and positions the decision surface in the form of a hyperplane between the two classes. The proof of convergence of the algorithm is known as the *perceptron convergence theorem*.

The perceptron built around a *single neuron* is limited to performing pattern classification with only two classes (hypotheses). By expanding the output (computation) layer of the perceptron to include more than one neuron, we may correspondingly perform classification with more than two classes. However, the classes have to be linearly separable for the perceptron to work properly. The important point is that insofar as the basic theory of the perceptron as a pattern classifier is concerned, we need consider only the case of a single neuron. The extension of the theory to the case of more than one neuron is trivial.

1.2 PERCEPTRON

Rosenblatt's perceptron is built around a nonlinear neuron, namely, the *McCulloch–Pitts model* of a neuron. From the introductory chapter we recall that such a neural modeling consists of a linear combiner followed by a hard limiter (performing the signum function), as depicted in Fig. 1.1. The summing node of the neural model computes a linear combination of the inputs applied to its synapses, as well as incorporates an externally applied bias. The resulting sum, that is, the induced local field, is applied to a hard

FIGURE 1.1 Signal-flow graph of the perceptron.



limiter. Accordingly, the neuron produces an output equal to $+1$ if the hard limiter input is positive, and -1 if it is negative.

In the signal-flow graph model of Fig. 1.1, the synaptic weights of the perceptron are denoted by $w_1, w_2, \dots, w_m$. Correspondingly, the inputs applied to the perceptron are denoted by $x_1, x_2, \dots, x_m$. The externally applied bias is denoted by b . From the model, we find that the hard limiter input, or induced local field, of the neuron is

$$v = \sum_{i=1}^m w_i x_i + b \quad (1.1)$$

The goal of the perceptron is to correctly classify the set of externally applied stimuli $x_1, x_2, \dots, x_m$ into one of two classes, $\mathcal{C}_1$ or $\mathcal{C}_2$. The decision rule for the classification is to assign the point represented by the inputs $x_1, x_2, \dots, x_m$ to class $\mathcal{C}_1$ if the perceptron output y is $+1$ and to class $\mathcal{C}_2$ if it is -1 .

To develop insight into the behavior of a pattern classifier, it is customary to plot a map of the decision regions in the m -dimensional signal space spanned by the m input variables $x_1, x_2, \dots, x_m$. In the simplest form of the perceptron, there are two decision regions separated by a *hyperplane*, which is defined by

$$\sum_{i=1}^m w_i x_i + b = 0 \quad (1.2)$$

This is illustrated in Fig. 1.2 for the case of two input variables x_1 and x_2 , for which the decision boundary takes the form of a straight line. A point (x_1, x_2) that lies above the boundary line is assigned to class $\mathcal{C}_1$, and a point (x_1, x_2) that lies below the boundary line is assigned to class $\mathcal{C}_2$. Note also that the effect of the bias b is merely to shift the decision boundary away from the origin.

The synaptic weights $w_1, w_2, \dots, w_m$ of the perceptron can be adapted on an iteration-by-iteration basis. For the adaptation, we may use an error-correction rule known as the perceptron convergence algorithm, discussed next.

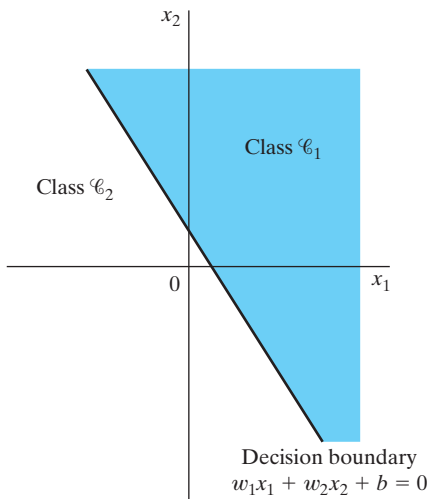


FIGURE 1.2 Illustration of the hyperplane (in this example, a straight line) as decision boundary for a two-dimensional, two-class pattern-classification problem.

1.3 THE PERCEPTRON CONVERGENCE THEOREM

To derive the error-correction learning algorithm for the perceptron, we find it more convenient to work with the modified signal-flow graph model in Fig. 1.3. In this second model, which is equivalent to that of Fig. 1.1, the bias $b(n)$ is treated as a synaptic weight driven by a fixed input equal to +1. We may thus define the $(m + 1)$ -by-1 input vector

$$\mathbf{x}(n) = [+1, x_1(n), x_2(n), \dots, x_m(n)]^T$$

where n denotes the time-step in applying the algorithm. Correspondingly, we define the $(m + 1)$ -by-1 weight vector as

$$\mathbf{w}(n) = [b, w_1(n), w_2(n), \dots, w_m(n)]^T$$

Accordingly, the linear combiner output is written in the compact form

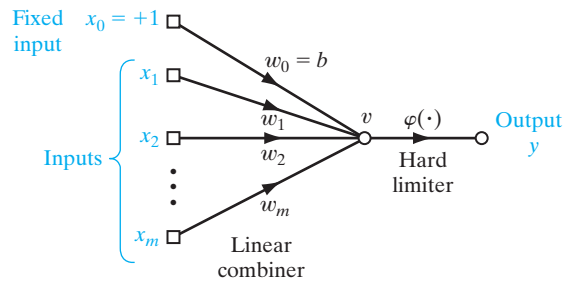
$$\begin{aligned} v(n) &= \sum_{i=0}^m w_i(n)x_i(n) \\ &= \mathbf{w}^T(n)\mathbf{x}(n) \end{aligned} \quad (1.3)$$

where, in the first line, $w_0(n)$, corresponding to $i = 0$, represents the bias b . For fixed n , the equation $\mathbf{w}^T \mathbf{x} = 0$, plotted in an m -dimensional space (and for some prescribed bias) with coordinates $x_1, x_2, \dots, x_m$, defines a hyperplane as the decision surface between two different classes of inputs.

For the perceptron to function properly, the two classes $\mathcal{C}_1$ and $\mathcal{C}_2$ must be *linearly separable*. This, in turn, means that the patterns to be classified must be sufficiently separated from each other to ensure that the decision surface consists of a hyperplane. This requirement is illustrated in Fig. 1.4 for the case of a two-dimensional perceptron. In Fig. 1.4a, the two classes $\mathcal{C}_1$ and $\mathcal{C}_2$ are sufficiently separated from each other for us to draw a hyperplane (in this case, a straight line) as the decision boundary. If, however, the two classes $\mathcal{C}_1$ and $\mathcal{C}_2$ are allowed to move too close to each other, as in Fig. 1.4b, they become nonlinearly separable, a situation that is beyond the computing capability of the perceptron.

Suppose then that the input variables of the perceptron originate from two linearly separable classes. Let $\mathcal{H}_1$ be the subspace of training vectors $\mathbf{x}_1(1), \mathbf{x}_1(2), \dots$ that belong to class $\mathcal{C}_1$, and let $\mathcal{H}_2$ be the subspace of training vectors $\mathbf{x}_2(1), \mathbf{x}_2(2), \dots$ that belong to class $\mathcal{C}_2$. The union of $\mathcal{H}_1$ and $\mathcal{H}_2$ is the complete space denoted by $\mathcal{H}$. Given the sets

FIGURE 1.3 Equivalent signal-flow graph of the perceptron; dependence on time has been omitted for clarity.



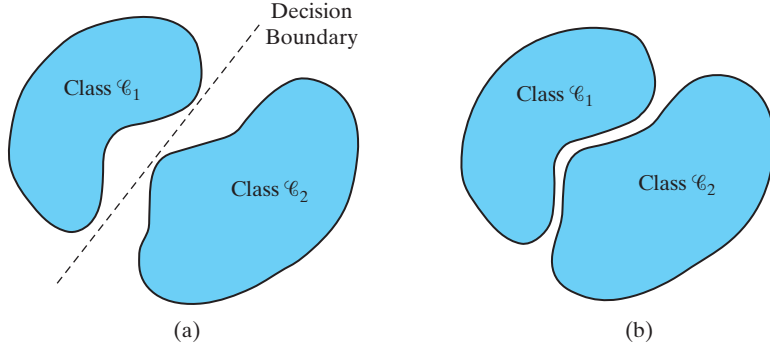


FIGURE 1.4 (a) A pair of linearly separable patterns. (b) A pair of non-linearly separable patterns.

of vectors $\mathcal{H}_1$ and $\mathcal{H}_2$ to train the classifier, the training process involves the adjustment of the weight vector $\mathbf{w}$ in such a way that the two classes $\mathcal{C}_1$ and $\mathcal{C}_2$ are linearly separable. That is, there exists a weight vector $\mathbf{w}$ such that we may state

$$\begin{aligned} \mathbf{w}^T \mathbf{x} &> 0 \text{ for every input vector } \mathbf{x} \text{ belonging to class } \mathcal{C}_1 \\ \mathbf{w}^T \mathbf{x} &\leq 0 \text{ for every input vector } \mathbf{x} \text{ belonging to class } \mathcal{C}_2 \end{aligned} \quad (1.4)$$

In the second line of Eq. (1.4), we have arbitrarily chosen to say that the input vector $\mathbf{x}$ belongs to class $\mathcal{C}_2$ if $\mathbf{w}^T \mathbf{x} = 0$. Given the subsets of training vectors $\mathcal{H}_1$ and $\mathcal{H}_2$, the training problem for the perceptron is then to find a weight vector $\mathbf{w}$ such that the two inequalities of Eq. (1.4) are satisfied.

The algorithm for adapting the weight vector of the elementary perceptron may now be formulated as follows:

1. If the n th member of the training set, $\mathbf{x}(n)$, is correctly classified by the weight vector $\mathbf{w}(n)$ computed at the n th iteration of the algorithm, no correction is made to the weight vector of the perceptron in accordance with the rule:

$$\begin{aligned} \mathbf{w}(n+1) &= \mathbf{w}(n) \text{ if } \mathbf{w}^T(n)\mathbf{x}(n) > 0 \text{ and } \mathbf{x}(n) \text{ belongs to class } \mathcal{C}_1 \\ \mathbf{w}(n+1) &= \mathbf{w}(n) \text{ if } \mathbf{w}^T(n)\mathbf{x}(n) \leq 0 \text{ and } \mathbf{x}(n) \text{ belongs to class } \mathcal{C}_2 \end{aligned} \quad (1.5)$$

2. Otherwise, the weight vector of the perceptron is updated in accordance with the rule

$$\begin{aligned} \mathbf{w}(n+1) &= \mathbf{w}(n) - \eta(n)\mathbf{x}(n) \text{ if } \mathbf{w}^T(n)\mathbf{x}(n) > 0 \text{ and } \mathbf{x}(n) \text{ belongs to class } \mathcal{C}_2 \\ \mathbf{w}(n+1) &= \mathbf{w}(n) + \eta(n)\mathbf{x}(n) \text{ if } \mathbf{w}^T(n)\mathbf{x}(n) \leq 0 \text{ and } \mathbf{x}(n) \text{ belongs to class } \mathcal{C}_1 \end{aligned} \quad (1.6)$$

where the *learning-rate parameter* $\eta(n)$ controls the adjustment applied to the weight vector at iteration n .

If $\eta(n) = \eta > 0$, where η is a constant independent of the iteration number n , then we have a *fixed-increment adaptation rule* for the perceptron.

In the sequel, we first prove the convergence of a fixed-increment adaptation rule for which $\eta = 1$. Clearly, the value of η is unimportant, so long as it is positive. A value

of $\eta \neq 1$ merely scales the pattern vectors without affecting their separability. The case of a variable $\eta(n)$ is considered later.

Proof of the perceptron convergence algorithm² is presented for the initial condition $\mathbf{w}(0) = \mathbf{0}$. Suppose that $\mathbf{w}^T(n)\mathbf{x}(n) < 0$ for $n = 1, 2, \dots$, and the input vector $\mathbf{x}(n)$ belongs to the subset $\mathcal{H}_1$. That is, the perceptron incorrectly classifies the vectors $\mathbf{x}(1), \mathbf{x}(2), \dots$, since the first condition of Eq. (1.4) is violated. Then, with the constant $\eta(n) = 1$, we may use the second line of Eq. (1.6) to write

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \mathbf{x}(n) \quad \text{for } \mathbf{x}(n) \text{ belonging to class } \mathcal{C}_1 \quad (1.7)$$

Given the initial condition $\mathbf{w}(0) = \mathbf{0}$, we may iteratively solve this equation for $\mathbf{w}(n+1)$, obtaining the result

$$\mathbf{w}(n+1) = \mathbf{x}(1) + \mathbf{x}(2) + \dots + \mathbf{x}(n) \quad (1.8)$$

Since the classes $\mathcal{C}_1$ and $\mathcal{C}_2$ are assumed to be linearly separable, there exists a solution $\mathbf{w}_o$ for which $\mathbf{w}_o^T \mathbf{x}(n) > 0$ for the vectors $\mathbf{x}(1), \dots, \mathbf{x}(n)$ belonging to the subset $\mathcal{H}_1$. For a fixed solution $\mathbf{w}_o$, we may then define a positive number α as

$$\alpha = \min_{\mathbf{x}(n) \in \mathcal{H}_1} \mathbf{w}_o^T \mathbf{x}(n) \quad (1.9)$$

Hence, multiplying both sides of Eq. (1.8) by the row vector $\mathbf{w}_o^T$, we get

$$\mathbf{w}_o^T \mathbf{w}(n+1) = \mathbf{w}_o^T \mathbf{x}(1) + \mathbf{w}_o^T \mathbf{x}(2) + \dots + \mathbf{w}_o^T \mathbf{x}(n)$$

Accordingly, in light of the definition given in Eq. (1.9), we have

$$\mathbf{w}_o^T \mathbf{w}(n+1) \geq n\alpha \quad (1.10)$$

Next we make use of an inequality known as the Cauchy–Schwarz inequality. Given two vectors $\mathbf{w}_o$ and $\mathbf{w}(n+1)$, the *Cauchy–Schwarz inequality* states that

$$\|\mathbf{w}_o\|^2 \|\mathbf{w}(n+1)\|^2 \geq [\mathbf{w}_o^T \mathbf{w}(n+1)]^2 \quad (1.11)$$

where $\|\cdot\|$ denotes the Euclidean norm of the enclosed argument vector, and the inner product $\mathbf{w}_o^T \mathbf{w}(n+1)$ is a scalar quantity. We now note from Eq. (1.10) that $[\mathbf{w}_o^T \mathbf{w}(n+1)]^2$ is equal to or greater than $n^2 \alpha^2$. From Eq. (1.11) we note that $\|\mathbf{w}_o\|^2 \|\mathbf{w}(n+1)\|^2$ is equal to or greater than $[\mathbf{w}_o^T \mathbf{w}(n+1)]^2$. It follows therefore that

$$\|\mathbf{w}_o\|^2 \|\mathbf{w}(n+1)\|^2 \geq n^2 \alpha^2$$

or, equivalently,

$$\|\mathbf{w}(n+1)\|^2 \geq \frac{n^2 \alpha^2}{\|\mathbf{w}_o\|^2} \quad (1.12)$$

We next follow another development route. In particular, we rewrite Eq. (1.7) in the form

$$\mathbf{w}(k+1) = \mathbf{w}(k) + \mathbf{x}(k) \quad \text{for } k = 1, \dots, n \quad \text{and} \quad \mathbf{x}(k) \in \mathcal{H}_1 \quad (1.13)$$

By taking the squared Euclidean norm of both sides of Eq. (1.13), we obtain

$$\|\mathbf{w}(k+1)\|^2 = \|\mathbf{w}(k)\|^2 + \|\mathbf{x}(k)\|^2 + 2\mathbf{w}^T(k)\mathbf{x}(k) \quad (1.14)$$

But, $\mathbf{w}^T(k)\mathbf{x}(k) \leq 0$. We therefore deduce from Eq. (1.14) that

$$\|\mathbf{w}(k+1)\|^2 \leq \|\mathbf{w}(k)\|^2 + \|\mathbf{x}(k)\|^2$$

or, equivalently,

$$\|\mathbf{w}(k+1)\|^2 - \|\mathbf{w}(k)\|^2 \leq \|\mathbf{x}(k)\|^2, \quad k = 1, \dots, n \quad (1.15)$$

Adding these inequalities for $k = 1, \dots, n$, and invoking the assumed initial condition $\mathbf{w}(0) = \mathbf{0}$, we get the inequality

$$\begin{aligned} \|\mathbf{w}(n+1)\|^2 &\leq \sum_{k=1}^n \|\mathbf{x}(k)\|^2 \\ &\leq n\beta \end{aligned} \quad (1.16)$$

where β is a positive number defined by

$$\beta = \max_{\mathbf{x}(k) \in \mathcal{H}_1} \|\mathbf{x}(k)\|^2 \quad (1.17)$$

Equation (1.16) states that the squared Euclidean norm of the weight vector $\mathbf{w}(n+1)$ grows at most linearly with the number of iterations n .

The second result of Eq. (1.16) is clearly in conflict with the earlier result of Eq. (1.12) for sufficiently large values of n . Indeed, we can state that n cannot be larger than some value $n_{\max}$ for which Eqs. (1.12) and (1.16) are both satisfied with the equality sign. That is, $n_{\max}$ is the solution of the equation

$$\frac{n_{\max}^2 \alpha^2}{\|\mathbf{w}_o\|^2} = n_{\max} \beta$$

Solving for $n_{\max}$, given a solution vector $\mathbf{w}_o$, we find that

$$n_{\max} = \frac{\beta \|\mathbf{w}_o\|^2}{\alpha^2} \quad (1.18)$$

We have thus proved that for $\eta(n) = 1$ for all n and $\mathbf{w}(0) = \mathbf{0}$, and given that a solution vector $\mathbf{w}_o$ exists, the rule for adapting the synaptic weights of the perceptron must terminate after at most $n_{\max}$ iterations. Surprisingly, this statement, proved for hypothesis $\mathcal{H}_1$, also holds for hypothesis $\mathcal{H}_2$. Note however,

We may now state the *fixed-increment convergence theorem* for the perceptron as follows (Rosenblatt, 1962):

Let the subsets of training vectors $\mathcal{H}_1$ and $\mathcal{H}_2$ be linearly separable. Let the inputs presented to the perceptron originate from these two subsets. The perceptron converges after some n_o iterations, in the sense that

$$\mathbf{w}(n_o) = \mathbf{w}(n_o + 1) = \mathbf{w}(n_o + 2) = \dots$$

is a solution vector for $n_0 \leq n_{\max}$.

Consider next the *absolute error-correction procedure* for the adaptation of a single-layer perceptron, for which $\eta(n)$ is variable. In particular, let $\eta(n)$ be the smallest integer for which the condition

$$\eta(n)\mathbf{x}^T(n)\mathbf{x}(n) > |\mathbf{w}^T(n)\mathbf{x}(n)|$$

holds. With this procedure we find that if the inner product $\mathbf{w}^T(n)\mathbf{x}(n)$ at iteration n has an incorrect sign, then $\mathbf{w}^T(n+1)\mathbf{x}(n)$ at iteration $n+1$ would have the correct sign. This suggests that if $\mathbf{w}^T(n)\mathbf{x}(n)$ has an incorrect sign, at iteration n , we may modify the training sequence at iteration $n+1$ by setting $\mathbf{x}(n+1) = \mathbf{x}(n)$. In other words, each pattern is presented repeatedly to the perceptron until that pattern is classified correctly.

Note also that the use of an initial value $\mathbf{w}(0)$ different from the null condition merely results in a decrease or increase in the number of iterations required to converge, depending on how $\mathbf{w}(0)$ relates to the solution $\mathbf{w}_o$. Regardless of the value assigned to $\mathbf{w}(0)$, the perceptron is assured of convergence.

In Table 1.1, we present a summary of the *perceptron convergence algorithm* (Lippmann, 1987). The symbol $\text{sgn}(\cdot)$, used in step 3 of the table for computing the actual response of the perceptron, stands for the *signum function*:

$$\text{sgn}(v) = \begin{cases} +1 & \text{if } v > 0 \\ -1 & \text{if } v < 0 \end{cases} \quad (1.19)$$

We may thus express the *quantized response* $y(n)$ of the perceptron in the compact form

$$y(n) = \text{sgn}[\mathbf{w}^T(n)\mathbf{x}(n)] \quad (1.20)$$

Notice that the input vector $\mathbf{x}(n)$ is an $(m+1)$ -by-1 vector whose first element is fixed at +1 throughout the computation. Correspondingly, the weight vector $\mathbf{w}(n)$ is an

TABLE 1.1 Summary of the Perceptron Convergence Algorithm

Variables and Parameters:

- $\mathbf{x}(n)$ = $(m+1)$ -by-1 input vector
 $= [+1, x_1(n), x_2(n), \dots, x_m(n)]^T$
- $\mathbf{w}(n)$ = $(m+1)$ -by-1 weight vector
 $= [b, w_1(n), w_2(n), \dots, w_m(n)]^T$
- b = bias
- $y(n)$ = actual response (quantized)
- $d(n)$ = desired response
- η = learning-rate parameter, a positive constant less than unity

1. *Initialization.* Set $\mathbf{w}(0) = \mathbf{0}$. Then perform the following computations for time-step $n = 1, 2, \dots$
2. *Activation.* At time-step n , activate the perceptron by applying continuous-valued input vector $\mathbf{x}(n)$ and desired response $d(n)$.
3. *Computation of Actual Response.* Compute the actual response of the perceptron as

$$y(n) = \text{sgn}[\mathbf{w}^T(n)\mathbf{x}(n)]$$

where $\text{sgn}(\cdot)$ is the signum function.

4. *Adaptation of Weight Vector.* Update the weight vector of the perceptron to obtain

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \eta[d(n) - y(n)]\mathbf{x}(n)$$

where

$$d(n) = \begin{cases} +1 & \text{if } \mathbf{x}(n) \text{ belongs to class } \mathcal{C}_1 \\ -1 & \text{if } \mathbf{x}(n) \text{ belongs to class } \mathcal{C}_2 \end{cases}$$

5. *Continuation.* Increment time step n by one and go back to step 2.

$(m + 1)$ -by-1 vector whose first element equals the bias b . One other important point to note in Table 1.1 is that we have introduced a *quantized desired response* $d(n)$, defined by

$$d(n) = \begin{cases} +1 & \text{if } \mathbf{x}(n) \text{ belongs to class } \mathcal{C}_1 \\ -1 & \text{if } \mathbf{x}(n) \text{ belongs to class } \mathcal{C}_2 \end{cases} \quad (1.21)$$

Thus, the adaptation of the weight vector $\mathbf{w}(n)$ is summed up nicely in the form of the *error-correction learning rule*

$$\mathbf{w}(n + 1) = \mathbf{w}(n) + \eta[d(n) - y(n)]\mathbf{x}(n) \quad (1.22)$$

where η is the *learning-rate parameter* and the difference $d(n) - y(n)$ plays the role of an *error signal*. The learning-rate parameter is a positive constant limited to the range $0 < \eta \leq 1$. When assigning a value to it inside this range, we must keep in mind two conflicting requirements (Lippmann, 1987):

- *averaging* of past inputs to provide stable weight estimates, which requires a small η ;
- *fast adaptation* with respect to real changes in the underlying distributions of the process responsible for the generation of the input vector $\mathbf{x}$, which requires a large η .

1.4 RELATION BETWEEN THE PERCEPTRON AND BAYES CLASSIFIER FOR A GAUSSIAN ENVIRONMENT

The perceptron bears a certain relationship to a classical pattern classifier known as the Bayes classifier. When the environment is Gaussian, the Bayes classifier reduces to a linear classifier. This is the same form taken by the perceptron. However, the linear nature of the perceptron is *not* contingent on the assumption of Gaussianity. In this section, we study this relationship and thereby develop further insight into the operation of the perceptron. We begin the discussion with a brief review of the Bayes classifier.

Bayes Classifier

In the *Bayes classifier*, or *Bayes hypothesis testing procedure*, we minimize the *average risk*, denoted by $\mathcal{R}$. For a two-class problem, represented by classes $\mathcal{C}_1$ and $\mathcal{C}_2$, the average risk is defined by Van Trees (1968) as

$$\begin{aligned} \mathcal{R} = & c_{11}p_1 \int_{\mathcal{H}_1} p_{\mathbf{x}}(\mathbf{x}|\mathcal{C}_1)d\mathbf{x} + c_{22}p_2 \int_{\mathcal{H}_2} p_{\mathbf{x}}(\mathbf{x}|\mathcal{C}_2)d\mathbf{x} \\ & + c_{21}p_1 \int_{\mathcal{H}_2} p_{\mathbf{x}}(\mathbf{x}|\mathcal{C}_1)d\mathbf{x} + c_{12}p_2 \int_{\mathcal{H}_1} p_{\mathbf{x}}(\mathbf{x}|\mathcal{C}_2)d\mathbf{x} \end{aligned} \quad (1.23)$$

where the various terms are defined as follows:

p_i = *prior probability* that the observation vector $\mathbf{x}$ (representing a realization of the random vector $\mathbf{X}$) corresponds to an object in class $\mathcal{C}_i$, with $i = 1, 2$, and $p_1 + p_2 = 1$

c_{ij} = cost of deciding in favor of class $\mathcal{C}_i$ represented by subspace $\mathcal{H}_i$ when class $\mathcal{C}_j$ is true (i.e., observation vector $\mathbf{x}$ corresponds to an object in class $\mathcal{C}_1$), with $i, j = 1, 2$

$p_{\mathbf{x}}(\mathbf{x}|\mathcal{C}_i)$ = conditional probability density function of the random vector $\mathbf{X}$, given that the observation vector $\mathbf{x}$ corresponds to an object in class $\mathcal{C}_1$, with $i = 1, 2$.

The first two terms on the right-hand side of Eq. (1.23) represent *correct* decisions (i.e., correct classifications), whereas the last two terms represent *incorrect* decisions (i.e., misclassifications). Each decision is weighted by the product of two factors: the cost involved in making the decision and the relative frequency (i.e., *prior* probability) with which it occurs.

The intention is to determine a strategy for the *minimum average risk*. Because we require that a decision be made, each observation vector $\mathbf{x}$ must be assigned in the overall observation space $\mathcal{X}$ to either $\mathcal{H}_1$ or $\mathcal{H}_2$. Thus,

$$\mathcal{X} = \mathcal{H}_1 + \mathcal{H}_2 \quad (1.24)$$

Accordingly, we may rewrite Eq. (1.23) in the equivalent form

$$\begin{aligned} \mathcal{R} = & c_{11}p_1 \int_{\mathcal{H}_1} p_{\mathbf{x}}(\mathbf{x}|\mathcal{C}_1) d\mathbf{x} + c_{22}p_2 \int_{\mathcal{H}-\mathcal{H}_1} p_{\mathbf{x}}(\mathbf{x}|\mathcal{C}_2) d\mathbf{x} \\ & + c_{21}p_1 \int_{\mathcal{H}-\mathcal{H}_1} p_{\mathbf{x}}(\mathbf{x}|\mathcal{C}_1) d\mathbf{x} + c_{12}p_2 \int_{\mathcal{H}_1} p_{\mathbf{x}}(\mathbf{x}|\mathcal{C}_2) d\mathbf{x} \end{aligned} \quad (1.25)$$

where $c_{11} < c_{21}$ and $c_{22} < c_{12}$. We now observe the fact that

$$\int_{\mathcal{X}} p_{\mathbf{x}}(\mathbf{x}|\mathcal{C}_1) d\mathbf{x} = \int_{\mathcal{X}} p_{\mathbf{x}}(\mathbf{x}|\mathcal{C}_2) d\mathbf{x} = 1 \quad (1.26)$$

Hence, Eq. (1.25) reduces to

$$\begin{aligned} \mathcal{R} = & c_{21}p_1 + c_{22}p_2 \\ & + \int_{\mathcal{H}_1} [p_2(c_{12} - c_{22}) p_{\mathbf{x}}(\mathbf{x}|\mathcal{C}_2) - p_1(c_{21} - c_{11}) p_{\mathbf{x}}(\mathbf{x}|\mathcal{C}_1)] d\mathbf{x} \end{aligned} \quad (1.27)$$

The first two terms on the right-hand side of Eq. (1.27) represent a fixed cost. Since the requirement is to minimize the average risk $\mathcal{R}$, we may therefore deduce the following strategy from Eq.(1.27) for optimum classification:

1. All values of the observation vector $\mathbf{x}$ for which the integrand (i.e., the expression inside the square brackets) is negative should be assigned to subset $\mathcal{H}_1$ (i.e., class $\mathcal{C}_1$), for the integral would then make a negative contribution to the risk $\mathcal{R}$.
2. All values of the observation vector $\mathbf{x}$ for which the integrand is positive should be excluded from subset $\mathcal{H}_1$ (i.e., assigned to class $\mathcal{C}_2$), for the integral would then make a positive contribution to the risk $\mathcal{R}$.
3. Values of $\mathbf{x}$ for which the integrand is zero have no effect on the average risk $\mathcal{R}$ and may be assigned arbitrarily. We shall assume that these points are assigned to subset $\mathcal{H}_2$ (i.e., class $\mathcal{C}_2$).

On this basis, we may now formulate the Bayes classifier as follows:

If the condition

$$p_1(c_{21} - c_{11}) p_{\mathbf{x}}(\mathbf{x}|\mathcal{C}_1) > p_2(c_{12} - c_{22}) p_{\mathbf{x}}(\mathbf{x}|\mathcal{C}_2)$$

holds, assign the observation vector $\mathbf{x}$ to subspace $\mathcal{X}_1$ (i.e., class $\mathcal{C}_1$). Otherwise assign $\mathbf{x}$ to $\mathcal{X}_2$ (i.e., class $\mathcal{C}_2$).

To simplify matters, define

$$\Lambda(\mathbf{x}) = \frac{p_{\mathbf{x}}(\mathbf{x}|\mathcal{C}_1)}{p_{\mathbf{x}}(\mathbf{x}|\mathcal{C}_2)} \quad (1.28)$$

and

$$\xi = \frac{p_2(c_{12} - c_{22})}{p_1(c_{21} - c_{11})} \quad (1.29)$$

The quantity $\Lambda(\mathbf{x})$, the ratio of two conditional probability density functions, is called the *likelihood ratio*. The quantity ξ is called the *threshold* of the test. Note that both $\Lambda(\mathbf{x})$ and ξ are always positive. In terms of these two quantities, we may now reformulate the Bayes classifier by stating the following

If, for an observation vector $\mathbf{x}$, the likelihood ratio $\Lambda(\mathbf{x})$ is greater than the threshold ξ , assign $\mathbf{x}$ to class $\mathcal{C}_1$. Otherwise, assign it to class $\mathcal{C}_2$.

Figure 1.5a depicts a block-diagram representation of the Bayes classifier. The important points in this block diagram are twofold:

1. The data processing involved in designing the Bayes classifier is confined entirely to the computation of the likelihood ratio $\Lambda(\mathbf{x})$.

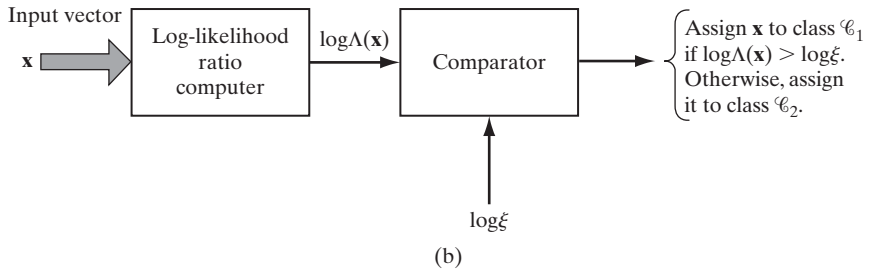
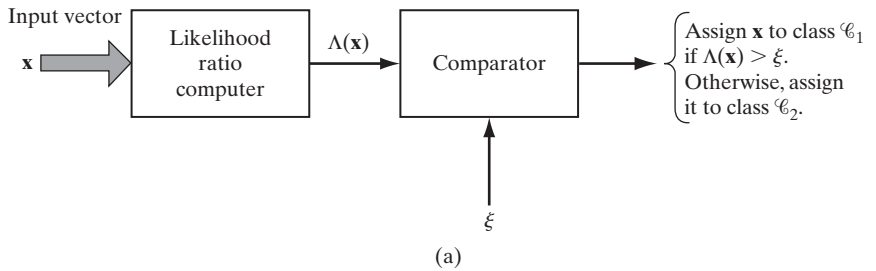


FIGURE 1.5 Two equivalent implementations of the Bayes classifier: (a) Likelihood ratio test, (b) Log-likelihood ratio test.

2. This computation is completely invariant to the values assigned to the prior probabilities and costs involved in the decision-making process. These quantities merely affect the value of the threshold ξ .

From a computational point of view, we find it more convenient to work with the logarithm of the likelihood ratio rather than the likelihood ratio itself. We are permitted to do this for two reasons. First, the logarithm is a monotonic function. Second, the likelihood ratio $\Lambda(\mathbf{x})$ and threshold ξ are both positive. Therefore, the Bayes classifier may be implemented in the equivalent form shown in Fig. 1.5b. For obvious reasons, the test embodied in this latter figure is called the *log-likelihood ratio test*.

Bayes Classifier for a Gaussian Distribution

Consider now the special case of a two-class problem, for which the underlying distribution is Gaussian. The random vector $\mathbf{X}$ has a mean value that depends on whether it belongs to class $\mathcal{C}_1$ or class $\mathcal{C}_2$, but the covariance matrix of $\mathbf{X}$ is the same for both classes. That is to say,

$$\begin{aligned} \text{Class } \mathcal{C}_1: \quad & \mathbb{E}[\mathbf{X}] = \boldsymbol{\mu}_1 \\ & \mathbb{E}[(\mathbf{X} - \boldsymbol{\mu}_1)(\mathbf{X} - \boldsymbol{\mu}_1)^T] = \mathbf{C} \\ \text{Class } \mathcal{C}_2: \quad & \mathbb{E}[\mathbf{X}] = \boldsymbol{\mu}_2 \\ & \mathbb{E}[(\mathbf{X} - \boldsymbol{\mu}_2)(\mathbf{X} - \boldsymbol{\mu}_2)^T] = \mathbf{C} \end{aligned}$$

The covariance matrix $\mathbf{C}$ is nondiagonal, which means that the samples drawn from classes $\mathcal{C}_1$ and $\mathcal{C}_2$ are *correlated*. It is assumed that $\mathbf{C}$ is nonsingular, so that its inverse matrix $\mathbf{C}^{-1}$ exists.

With this background, we may express the conditional probability density function of $\mathbf{X}$ as the multivariate Gaussian distribution

$$p_{\mathbf{X}}(\mathbf{x}|\mathcal{C}_i) = \frac{1}{(2\pi)^{m/2}(\det(\mathbf{C}))^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_i)^T \mathbf{C}^{-1}(\mathbf{x} - \boldsymbol{\mu}_i)\right), \quad i = 1, 2 \quad (1.30)$$

where m is the dimensionality of the observation vector $\mathbf{x}$.

We further assume the following:

1. The two classes $\mathcal{C}_1$ and $\mathcal{C}_2$ are equiprobable:

$$p_1 = p_2 = \frac{1}{2} \quad (1.31)$$

2. Misclassifications carry the same cost, and no cost is incurred on correct classifications:

$$c_{21} = c_{12} \quad \text{and} \quad c_{11} = c_{22} = 0 \quad (1.32)$$

We now have the information we need to design the Bayes classifier for the two-class problem. Specifically, by substituting Eq. (1.30) into (1.28) and taking the natural logarithm, we get (after simplifications)

$$\begin{aligned}\log \Lambda(\mathbf{x}) &= -\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_1)^T \mathbf{C}^{-1}(\mathbf{x} - \boldsymbol{\mu}_1) + \frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_2)^T \mathbf{C}^{-1}(\mathbf{x} - \boldsymbol{\mu}_2) \\ &= (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)^T \mathbf{C}^{-1} \mathbf{x} + \frac{1}{2}(\boldsymbol{\mu}_2^T \mathbf{C}^{-1} \boldsymbol{\mu}_2 - \boldsymbol{\mu}_1^T \mathbf{C}^{-1} \boldsymbol{\mu}_1)\end{aligned}\quad (1.33)$$

By substituting Eqs. (1.31) and (1.32) into Eq. (1.29) and taking the natural logarithm, we get

$$\log \xi = 0 \quad (1.34)$$

Equations (1.33) and (1.34) state that the Bayes classifier for the problem at hand is a *linear classifier*, as described by the relation

$$y = \mathbf{w}^T \mathbf{x} + b \quad (1.35)$$

where

$$\mathbf{y} = \log \Lambda(\mathbf{x}) \quad (1.36)$$

$$\mathbf{w} = \mathbf{C}^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2) \quad (1.37)$$

$$b = \frac{1}{2}(\boldsymbol{\mu}_2^T \mathbf{C}^{-1} \boldsymbol{\mu}_2 - \boldsymbol{\mu}_1^T \mathbf{C}^{-1} \boldsymbol{\mu}_1) \quad (1.38)$$

More specifically, the classifier consists of a linear combiner with weight vector $\mathbf{w}$ and bias b , as shown in Fig. 1.6.

On the basis of Eq. (1.35), we may now describe the log-likelihood ratio test for our two-class problem as follows:

If the output y of the linear combiner (including the bias b) is positive, assign the observation vector $\mathbf{x}$ to class $\mathcal{C}_1$. Otherwise, assign it to class $\mathcal{C}_2$.

The operation of the Bayes classifier for the Gaussian environment described herein is analogous to that of the perceptron in that they are both linear classifiers; see Eqs. (1.1) and (1.35). There are, however, some subtle and important differences between them, which should be carefully examined (Lippmann, 1987):

- The perceptron operates on the premise that the patterns to be classified are *linearly separable*. The Gaussian distributions of the two patterns assumed in the derivation of the Bayes classifier certainly do *overlap* each other and are therefore *not separable*. The extent of the overlap is determined by the mean vectors

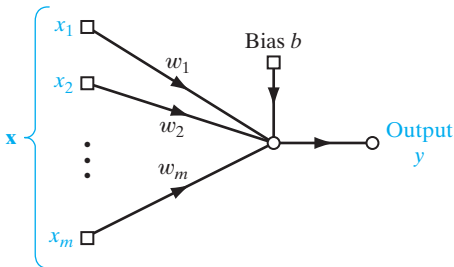
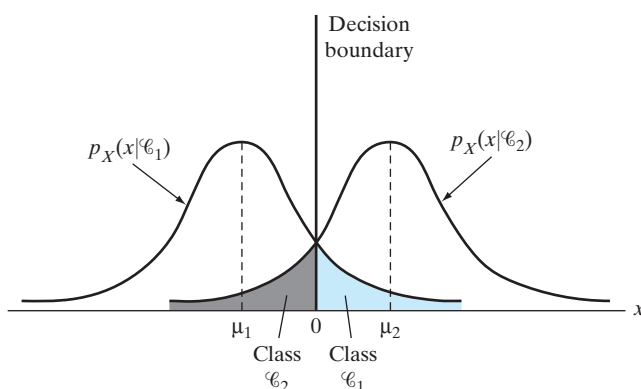


FIGURE 1.6 Signal-flow graph of Gaussian classifier.

FIGURE 1.7 Two overlapping, one-dimensional Gaussian distributions.



μ_1 and μ_2 and the covariance matrix $\mathbf{C}$. The nature of this overlap is illustrated in Fig. 1.7 for the special case of a scalar random variable (i.e., dimensionality $m = 1$). When the inputs are nonseparable and their distributions overlap as illustrated, the perceptron convergence algorithm develops a problem because decision boundaries between the different classes may oscillate continuously.

- The Bayes classifier minimizes the probability of classification error. This minimization is independent of the overlap between the underlying Gaussian distributions of the two classes. For example, in the special case illustrated in Fig. 1.7, the Bayes classifier always positions the decision boundary at the point where the Gaussian distributions for the two classes $\mathcal{C}_1$ and $\mathcal{C}_2$ cross each other.
- The perceptron convergence algorithm is *nonparametric* in the sense that it makes no assumptions concerning the form of the underlying distributions. It operates by concentrating on errors that occur where the distributions overlap. It may therefore work well when the inputs are generated by nonlinear physical mechanisms and when their distributions are heavily skewed and non-Gaussian. In contrast, the Bayes classifier is *parametric*; its derivation is contingent on the assumption that the underlying distributions be Gaussian, which may limit its area of application.
- The perceptron convergence algorithm is both adaptive and simple to implement; its storage requirement is confined to the set of synaptic weights and bias. On the other hand, the design of the Bayes classifier is fixed; it can be made adaptive, but at the expense of increased storage requirements and more complex computations.

1.5 COMPUTER EXPERIMENT: PATTERN CLASSIFICATION

The objective of this computer experiment is twofold:

- (i) to lay down the specifications of a *double-moon classification problem* that will serve as the basis of a prototype for the part of the book that deals with pattern-classification experiments;

6. Section 4.17 describes an important neural network structure: the *convolutional multilayer perceptron*. This network has been successfully used in the solution of difficult pattern-recognition problems.
7. Section 4.18 deals with nonlinear filtering, where time plays a key role. The discussion begins with short-term memory structures, setting the stage for the universal myopic mapping theorem.
8. Section 4.19 discusses the issue of small-scale versus large-scale learning problems.

The chapter concludes with summary and discussion in Section 4.20.

4.1 INTRODUCTION

In Chapter 1, we studied Rosenblatt's perceptron, which is basically a single-layer neural network. Therein, we showed that this network is limited to the classification of linearly separable patterns. Then we studied adaptive filtering in Chapter 3, using Widrow and Hoff's LMS algorithm. This algorithm is also based on a single linear neuron with adjustable weights, which limits the computing power of the algorithm. To overcome the practical limitations of the perceptron and the LMS algorithm, we look to a neural network structure known as the *multilayer perceptron*.

The following three points highlight the basic features of multilayer perceptrons:

- The model of each neuron in the network includes a nonlinear activation function that is *differentiable*.
- The network contains one or more layers that are *hidden* from both the input and output nodes.
- The network exhibits a high degree of *connectivity*, the extent of which is determined by synaptic weights of the network.

These same characteristics, however, are also responsible for the deficiencies in our knowledge on the behavior of the network. First, the presence of a distributed form of nonlinearity and the high connectivity of the network make the theoretical analysis of a multilayer perceptron difficult to undertake. Second, the use of hidden neurons makes the learning process harder to visualize. In an implicit sense, the learning process must decide which features of the input pattern should be represented by the hidden neurons. The learning process is therefore made more difficult because the search has to be conducted in a much larger space of possible functions, and a choice has to be made between alternative representations of the input pattern.

A popular method for the training of multilayer perceptrons is the back-propagation algorithm, which includes the LMS algorithm as a special case. The training proceeds in two phases:

1. In the *forward phase*, the synaptic weights of the network are fixed and the input signal is propagated through the network, layer by layer, until it reaches the output. Thus, in this phase, changes are confined to the activation potentials and outputs of the neurons in the network.

2. In the *backward phase*, an error signal is produced by comparing the output of the network with a desired response. The resulting error signal is propagated through the network, again layer by layer, but this time the propagation is performed in the backward direction. In this second phase, successive adjustments are made to the synaptic weights of the network. Calculation of the adjustments for the output layer is straightforward, but it is much more challenging for the hidden layers.

Usage of the term “back propagation” appears to have evolved after 1985, when the term was popularized through the publication of the seminal book entitled *Parallel Distributed Processing* (Rumelhart and McClelland, 1986).

The development of the back-propagation algorithm in the mid-1980s represented a landmark in neural networks in that it provided a *computationally efficient* method for the training of multilayer perceptrons, putting to rest the pessimism about learning in multilayer perceptrons that may have been inferred from the book by Minsky and Papert (1969).

4.2 SOME PRELIMINARIES

Figure 4.1 shows the architectural graph of a multilayer perceptron with two hidden layers and an output layer. To set the stage for a description of the multilayer perceptron in its general form, the network shown here is *fully connected*. This means that a neuron in any layer of the network is connected to all the neurons (nodes) in the previous layer. Signal flow through the network progresses in a forward direction, from left to right and on a layer-by-layer basis.

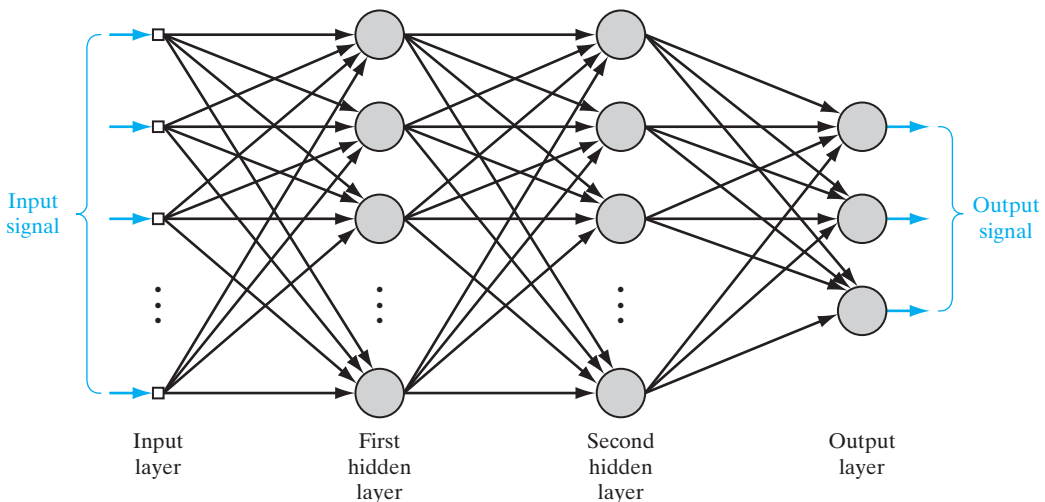


FIGURE 4.1 Architectural graph of a multilayer perceptron with two hidden layers.

Figure 4.2 depicts a portion of the multilayer perceptron. Two kinds of signals are identified in this network:

1. **Function Signals.** A function signal is an input signal (stimulus) that comes in at the input end of the network, propagates forward (neuron by neuron) through the network, and emerges at the output end of the network as an output signal. We refer to such a signal as a “function signal” for two reasons. First, it is presumed to perform a useful function at the output of the network. Second, at each neuron of the network through which a function signal passes, the signal is calculated as a function of the inputs and associated weights applied to that neuron. The function signal is also referred to as the input signal.
2. **Error Signals.** An error signal originates at an output neuron of the network and propagates backward (layer by layer) through the network. We refer to it as an “error signal” because its computation by every neuron of the network involves an error-dependent function in one form or another.

The output neurons constitute the output layer of the network. The remaining neurons constitute hidden layers of the network. Thus, the hidden units are not part of the output or input of the network—hence their designation as “hidden.” The first hidden layer is fed from the input layer made up of sensory units (source nodes); the resulting outputs of the first hidden layer are in turn applied to the next hidden layer; and so on for the rest of the network.

Each hidden or output neuron of a multilayer perceptron is designed to perform two computations:

1. the computation of the function signal appearing at the output of each neuron, which is expressed as a continuous nonlinear function of the input signal and synaptic weights associated with that neuron;
2. the computation of an estimate of the gradient vector (i.e., the gradients of the error surface with respect to the weights connected to the inputs of a neuron), which is needed for the backward pass through the network.

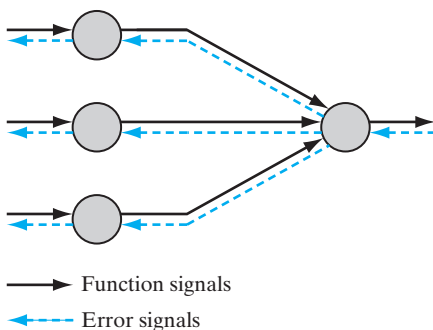


FIGURE 4.2 Illustration of the directions of two basic signal flows in a multilayer perceptron: forward propagation of function signals and back propagation of error signals.

Function of the Hidden Neurons

The hidden neurons act as *feature detectors*; as such, they play a critical role in the operation of a multilayer perceptron. As the learning process progresses across the multilayer perceptron, the hidden neurons begin to gradually “discover” the salient features that characterize the training data. They do so by performing a nonlinear transformation on the input data into a new space called the *feature space*. In this new space, the classes of interest in a pattern-classification task, for example, may be more easily separated from each other than could be the case in the original input data space. Indeed, it is the formation of this feature space through supervised learning that distinguishes the multilayer perceptron from Rosenblatt’s perceptron.

Credit-Assignment Problem

When studying learning algorithms for distributed systems, exemplified by the multilayer perceptron of Figure 4.1, it is instructive to pay attention to the notion of *credit assignment*. Basically, the credit-assignment problem is the problem of assigning *credit* or *blame* for overall outcomes to each of the *internal decisions* made by the hidden computational units of the distributed learning system, recognizing that those decisions are responsible for the overall outcomes in the first place.

In a multilayer perceptron using *error-correlation learning*, the credit-assignment problem arises because the operation of each hidden neuron and of each output neuron in the network is important to the network’s correct overall action on a learning task of interest. That is, in order to solve the prescribed task, the network must assign certain forms of behavior to all of its neurons through a specification of the error-correction learning algorithm. With this background, consider the multilayer perceptron depicted in Fig. 4.1. Since each output neuron is visible to the outside world, it is possible to supply a desired response to guide the behavior of such a neuron. Thus, as far as output neurons are concerned, it is a straightforward matter to adjust the synaptic weights of each output neuron in accordance with the error-correction algorithm. But how do we assign credit or blame for the action of the hidden neurons when the error-correction learning algorithm is used to adjust the respective synaptic weights of these neurons? The answer to this fundamental question requires more detailed attention than in the case of output neurons.

In what follows in this chapter, we show that the back-propagation algorithm, basic to the training of a multilayer perceptron, solves the credit-assignment problem in an elegant manner. But before proceeding to do that, we describe two basic methods of supervised learning in the next section.

4.3 BATCH LEARNING AND ON-LINE LEARNING

Consider a multilayer perceptron with an input layer of source nodes, one or more hidden layers, and an output layer consisting of one or more neurons; as illustrated in Fig. 4.1. Let

$$\mathcal{T} = \{\mathbf{x}(n), \mathbf{d}(n)\}_{n=1}^N \quad (4.1)$$

denote the *training sample* used to train the network in a supervised manner. Let $y_j(n)$ denote the function signal produced at the output of neuron j in the output layer by the stimulus $\mathbf{x}(n)$ applied to the input layer. Correspondingly, the *error signal* produced at the output of neuron j is defined by

$$e_j(n) = d_j(n) - y_j(n) \quad (4.2)$$

where $d_j(n)$ is the i th element of the desired-response vector $\mathbf{d}(n)$. Following the terminology of the LMS algorithm studied in Chapter 3, the *instantaneous error energy* of neuron j is defined by

$$\mathcal{E}_j(n) = \frac{1}{2} e_j^2(n) \quad (4.3)$$

Summing the error-energy contributions of all the neurons in the output layer, we express the *total instantaneous error energy* of the whole network as

$$\begin{aligned} \mathcal{E}(n) &= \sum_{j \in C} \mathcal{E}_j(n) \\ &= \frac{1}{2} \sum_{j \in C} e_j^2(n) \end{aligned} \quad (4.4)$$

where the set C includes all the neurons in the output layer. With the training sample consisting of N examples, the *error energy averaged over the training sample*, or the *empirical risk*, is defined by

$$\begin{aligned} \mathcal{E}_{\text{av}}(N) &= \frac{1}{N} \sum_{n=1}^N \mathcal{E}(n) \\ &= \frac{1}{2N} \sum_{n=1}^N \sum_{j \in C} e_j^2(n) \end{aligned} \quad (4.5)$$

Naturally, the instantaneous error energy, and therefore the average error energy, are both functions of all the adjustable synaptic weights (i.e., free parameters) of the multilayer perceptron. This functional dependence has not been included in the formulas for $\mathcal{E}(n)$ and $\mathcal{E}_{\text{av}}(N)$, merely to simplify the terminology.

Depending on how the supervised learning of the multilayer perceptron is actually performed, we may identify two different methods—namely, batch learning and on-line learning, as discussed next in the context of gradient descent.

Batch Learning

In the batch method of supervised learning, adjustments to the synaptic weights of the multilayer perceptron are performed *after* the presentation of *all* the N examples in the training sample $\mathcal{T}$ that constitute one *epoch* of training. In other words, the cost function for batch learning is defined by the average error energy $\mathcal{E}_{\text{av}}$. Adjustments to the synaptic weights of the multilayer perceptron are made on an *epoch-by-epoch basis*. Correspondingly, one realization of the learning curve is obtained by plotting $\mathcal{E}_{\text{av}}$ versus the number

of epochs, where, for each epoch of training, the examples in the training sample $\mathcal{T}$ are *randomly shuffled*. The learning curve is then computed by *ensemble averaging* a large enough number of such realizations, where each realization is performed for a *different set of initial conditions* chosen at random.

With the method of gradient descent used to perform the training, the advantages of batch learning include the following:

- *accurate estimation* of the gradient vector (i.e., the derivative of the cost function $\mathcal{E}_{av}$ with respect to the weight vector $\mathbf{w}$), thereby guaranteeing, under simple conditions, convergence of the method of steepest descent to a local minimum;
- *parallelization* of the learning process.

However, from a practical perspective, batch learning is rather demanding in terms of *storage requirements*.

In a statistical context, batch learning may be viewed as a form of *statistical inference*. It is therefore well suited for solving *nonlinear regression problems*.

On-line Learning

In the on-line method of supervised learning, adjustments to the synaptic weights of the multilayer perceptron are performed on an *example-by-example basis*. The cost function to be minimized is therefore the total instantaneous error energy $\mathcal{E}(n)$.

Consider an epoch of N training examples arranged in the order $\{\mathbf{x}(1), \mathbf{d}(1)\}, \{\mathbf{x}(2), \mathbf{d}(2)\}, \dots, \{\mathbf{x}(N), \mathbf{d}(N)\}$. The first example pair $\{\mathbf{x}(1), \mathbf{d}(1)\}$ in the epoch is presented to the network, and the weight adjustments are performed using the method of gradient descent. Then the second example $\{\mathbf{x}(2), \mathbf{d}(2)\}$ in the epoch is presented to the network, which leads to further adjustments to weights in the network. This procedure is continued until the last example $\{\mathbf{x}(N), \mathbf{d}(N)\}$ is accounted for. Unfortunately, such a procedure works against the parallelization of on-line learning.

For a given set of initial conditions, a single realization of the learning curve is obtained by plotting the final value $\mathcal{E}(N)$ versus the number of epochs used in the training session, where, as before, the training examples are randomly shuffled after each epoch. As with batch learning, the learning curve for on-line learning is computed by ensemble averaging such realizations over a large enough number of initial conditions chosen at random. Naturally, for a given network structure, the learning curve obtained under on-line learning will be quite different from that under batch learning.

Given that the training examples are presented to the network in a random manner, the use of on-line learning makes the search in the multidimensional weight space *stochastic* in nature; it is for this reason that the method of on-line learning is sometimes referred to as a *stochastic method*. This stochasticity has the desirable effect of making it less likely for the learning process to be trapped in a local minimum, which is a definite advantage of on-line learning over batch learning. Another advantage of on-line learning is the fact that it requires much less storage than batch learning.

Moreover, when the training data are *redundant* (i.e., the training sample $\mathcal{T}$ contains several copies of the same example), we find that, unlike batch learning, on-line

learning is able to take advantage of this redundancy because the examples are presented one at a time.

Another useful property of on-line learning is its ability to *track small changes* in the training data, particularly when the environment responsible for generating the data is nonstationary.

To summarize, despite the disadvantages of on-line learning, it is highly popular for solving *pattern-classification problems* for two important practical reasons:

- On-line learning is simple to implement.
- It provides effective solutions to large-scale and difficult pattern-classification problems.

It is for these two reasons that much of the material presented in this chapter is devoted to on-line learning.

4.4 THE BACK-PROPAGATION ALGORITHM

The popularity of on-line learning for the supervised training of multilayer perceptrons has been further enhanced by the development of the back-propagation algorithm. To describe this algorithm, consider Fig. 4.3, which depicts neuron j being fed by a set of

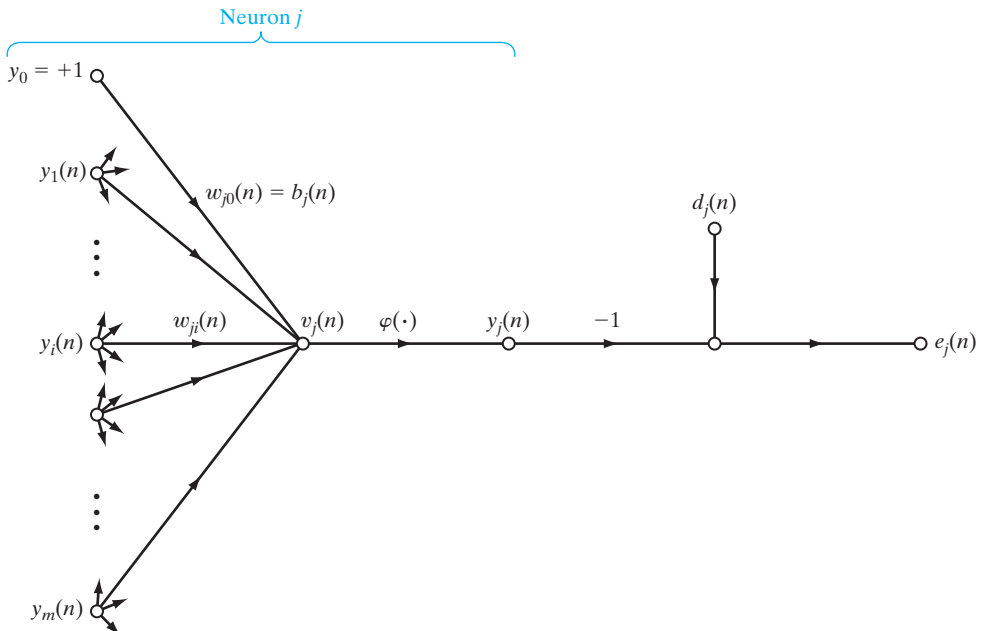


FIGURE 4.3 Signal-flow graph highlighting the details of output neuron j .

function signals produced by a layer of neurons to its left. The induced local field $v_j(n)$ produced at the input of the activation function associated with neuron j is therefore

$$v_j(n) = \sum_{i=0}^m w_{ji}(n) y_i(n) \quad (4.6)$$

where m is the total number of inputs (excluding the bias) applied to neuron j . The synaptic weight w_{j0} (corresponding to the fixed input $y_0 = +1$) equals the bias b_j applied to neuron j . Hence, the function signal $y_j(n)$ appearing at the output of neuron j at iteration n is

$$y_j(n) = \varphi_j(v_j(n)) \quad (4.7)$$

In a manner similar to the LMS algorithm studied in Chapter 3, the back-propagation algorithm applies a correction $\Delta w_{ji}(n)$ to the synaptic weight $w_{ji}(n)$, which is proportional to the partial derivative $\partial \mathcal{E}(n)/\partial w_{ji}(n)$. According to the *chain rule* of calculus, we may express this gradient as

$$\frac{\partial \mathcal{E}(n)}{\partial w_{ji}(n)} = \frac{\partial \mathcal{E}(n)}{\partial e_j(n)} \frac{\partial e_j(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)} \frac{\partial v_j(n)}{\partial w_{ji}(n)} \quad (4.8)$$

The partial derivative $\partial \mathcal{E}(n)/\partial w_{ji}(n)$ represents a *sensitivity factor*, determining the direction of search in weight space for the synaptic weight w_{ji} .

Differentiating both sides of Eq. (4.4) with respect to $e_j(n)$, we get

$$\frac{\partial \mathcal{E}(n)}{\partial e_j(n)} = e_j(n) \quad (4.9)$$

Differentiating both sides of Eq. (4.2) with respect to $y_j(n)$, we get

$$\frac{\partial e_j(n)}{\partial y_j(n)} = -1 \quad (4.10)$$

Next, differentiating Eq. (4.7) with respect to $v_j(n)$, we get

$$\frac{\partial y_j(n)}{\partial v_j(n)} = \varphi'_j(v_j(n)) \quad (4.11)$$

where the use of prime (on the right-hand side) signifies differentiation with respect to the argument. Finally, differentiating Eq. (4.6) with respect to $w_{ji}(n)$ yields

$$\frac{\partial v_j(n)}{\partial w_{ji}(n)} = y_i(n) \quad (4.12)$$

The use of Eqs. (4.9) to (4.12) in Eq. (4.8) yields

$$\frac{\partial \mathcal{E}(n)}{\partial w_{ji}(n)} = -e_j(n) \varphi'_j(v_j(n)) y_i(n) \quad (4.13)$$

The correction $\Delta w_{ji}(n)$ applied to $w_{ji}(n)$ is defined by the *delta rule*, or

$$\Delta w_{ji}(n) = -\eta \frac{\partial \mathcal{E}(n)}{\partial w_{ji}(n)} \quad (4.14)$$

where η is the *learning-rate parameter* of the back-propagation algorithm. The use of the minus sign in Eq. (4.14) accounts for *gradient descent* in weight space (i.e., seeking a direction for weight change that reduces the value of $\mathcal{E}(n)$). Accordingly, the use of Eq. (4.13) in Eq. (4.14) yields

$$\Delta w_{ji}(n) = \eta \delta_j(n) y_i(n) \quad (4.15)$$

where the *local gradient* $\delta_j(n)$ is defined by

$$\begin{aligned} \delta_j(n) &= \frac{\partial \mathcal{E}(n)}{\partial v_j(n)} \\ &= \frac{\partial \mathcal{E}(n)}{\partial e_j(n)} \frac{\partial e_j(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)} \\ &= e_j(n) \phi'_j(v_j(n)) \end{aligned} \quad (4.16)$$

The local gradient points to required changes in synaptic weights. According to Eq. (4.16), the local gradient $\delta_j(n)$ for output neuron j is equal to the product of the corresponding error signal $e_j(n)$ for that neuron and the derivative $\phi'_j(v_j(n))$ of the associated activation function.

From Eqs. (4.15) and (4.16), we note that a key factor involved in the calculation of the weight adjustment $\Delta w_{ji}(n)$ is the error signal $e_j(n)$ at the output of neuron j . In this context, we may identify two distinct cases, depending on where in the network neuron j is located. In case 1, neuron j is an output node. This case is simple to handle because each output node of the network is supplied with a desired response of its own, making it a straightforward matter to calculate the associated error signal. In case 2, neuron j is a hidden node. Even though hidden neurons are not directly accessible, they share responsibility for any error made at the output of the network. The question, however, is to know how to penalize or reward hidden neurons for their share of the responsibility. This problem is the *credit-assignment problem* considered in Section 4.2.

Case 1 Neuron j Is an Output Node

When neuron j is located in the output layer of the network, it is supplied with a desired response of its own. We may use Eq. (4.2) to compute the error signal $e_j(n)$ associated with this neuron; see Fig. 4.3. Having determined $e_j(n)$, we find it a straightforward matter to compute the local gradient $\delta_j(n)$ by using Eq. (4.16).

Case 2 Neuron j Is a Hidden Node

When neuron j is located in a hidden layer of the network, there is no specified desired response for that neuron. Accordingly, the error signal for a hidden neuron would have to be determined recursively and working backwards in terms of the error signals of all the neurons to which that hidden neuron is directly connected; this is where the

development of the back-propagation algorithm gets complicated. Consider the situation in Fig. 4.4, which depicts neuron j as a hidden node of the network. According to Eq. (4.16), we may redefine the local gradient $\delta_j(n)$ for hidden neuron j as

$$\begin{aligned}\delta_j(n) &= -\frac{\partial \mathcal{E}(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)} \\ &= -\frac{\partial \mathcal{E}(n)}{\partial y_j(n)} \phi'_j(v_j(n)), \quad \text{neuron } j \text{ is hidden}\end{aligned}\quad (4.17)$$

where in the second line we have used Eq. (4.11). To calculate the partial derivative $\partial \mathcal{E}(n)/\partial y_j(n)$, we may proceed as follows: From Fig. 4.4, we see that

$$\mathcal{E}(n) = \frac{1}{2} \sum_{k \in C} e_k^2(n), \quad \text{neuron } k \text{ is an output node} \quad (4.18)$$

which is Eq. (4.4) with index k used in place of index j . We have made this substitution in order to avoid confusion with the use of index j that refers to a hidden neuron under case 2. Differentiating Eq. (4.18) with respect to the function signal $y_j(n)$, we get

$$\frac{\partial \mathcal{E}(n)}{\partial y_j(n)} = \sum_k e_k \frac{\partial e_k(n)}{\partial y_j(n)} \quad (4.19)$$

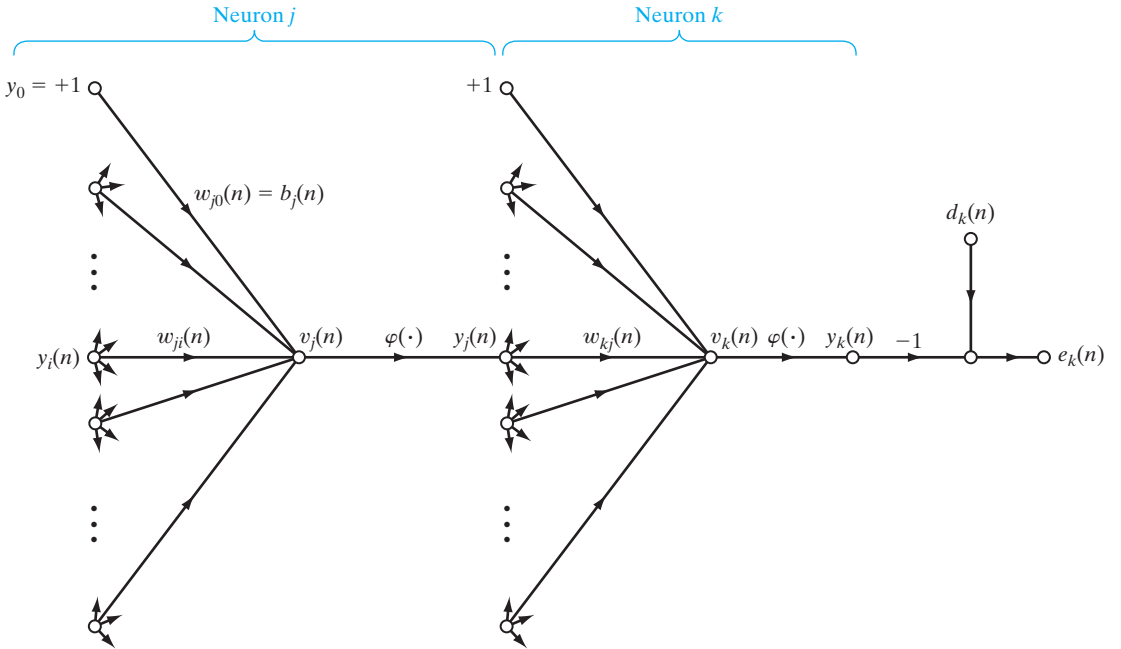


FIGURE 4.4 Signal-flow graph highlighting the details of output neuron k connected to hidden neuron j .

Next we use the chain rule for the partial derivative $\partial e_k(n)/\partial y_j(n)$ and rewrite Eq. (4.19) in the equivalent form

$$\frac{\partial \mathcal{E}(n)}{\partial y_j(n)} = \sum_k e_k(n) \frac{\partial e_k(n)}{\partial v_k(n)} \frac{\partial v_k(n)}{\partial y_j(n)} \quad (4.20)$$

However, from Fig. 4.4, we note that

$$\begin{aligned} e_k(n) &= d_k(n) - y_k(n) \\ &= d_k(n) - \varphi_k(v_k(n)), \quad \text{neuron } k \text{ is an output node} \end{aligned} \quad (4.21)$$

Hence,

$$\frac{\partial e_k(n)}{\partial v_k(n)} = -\varphi'_k(v_k(n)) \quad (4.22)$$

We also note from Fig. 4.4 that for neuron k , the induced local field is

$$v_k(n) = \sum_{j=0}^m w_{kj}(n) y_j(n) \quad (4.23)$$

where m is the total number of inputs (excluding the bias) applied to neuron k . Here again, the synaptic weight $w_{k0}(n)$ is equal to the bias $b_k(n)$ applied to neuron k , and the corresponding input is fixed at the value $+1$. Differentiating Eq. (4.23) with respect to $y_j(n)$ yields

$$\frac{\partial v_k(n)}{\partial y_j(n)} = w_{kj}(n) \quad (4.24)$$

By using Eqs. (4.22) and (4.24) in Eq. (4.20), we get the desired partial derivative

$$\begin{aligned} \frac{\partial \mathcal{E}(n)}{\partial y_j(n)} &= - \sum_k e_k(n) \varphi'_k(v_k(n)) w_{kj}(n) \\ &= - \sum_k \delta_k(n) w_{kj}(n) \end{aligned} \quad (4.25)$$

where, in the second line, we have used the definition of the local gradient $\delta_k(n)$ given in Eq. (4.16), with the index k substituted for j .

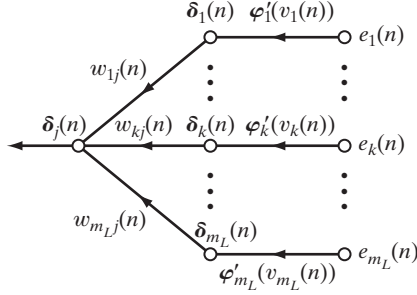
Finally, using Eq. (4.25) in Eq. (4.17), we get the *back-propagation formula* for the local gradient $\delta_j(n)$, described by

$$\delta_j(n) = \varphi'_j(v_j(n)) \sum_k \delta_k(n) w_{kj}(n), \quad \text{neuron } j \text{ is hidden} \quad (4.26)$$

Figure 4.5 shows the signal-flow graph representation of Eq. (4.26), assuming that the output layer consists of m_L neurons.

The outside factor $\varphi'_j(v_j(n))$ involved in the computation of the local gradient $\delta_j(n)$ in Eq. (4.26) depends solely on the activation function associated with hidden neuron j . The remaining factor involved in this computation—namely, the summation over k —depends on two sets of terms. The first set of terms, the $\delta_k(n)$, requires knowledge of the error

FIGURE 4.5 Signal-flow graph of a part of the adjoint system pertaining to back-propagation of error signals.



signals $e_k(n)$ for all neurons that lie in the layer to the immediate right of hidden neuron j and that are directly connected to neuron j ; see Fig. 4.4. The second set of terms, the $w_{kj}(n)$, consists of the synaptic weights associated with these connections.

We now summarize the relations that we have derived for the back-propagation algorithm. First, the correction $\Delta w_{ji}(n)$ applied to the synaptic weight connecting neuron i to neuron j is defined by the delta rule:

$$\begin{pmatrix} \text{Weight} \\ \text{correction} \\ \Delta w_{ji}(n) \end{pmatrix} = \begin{pmatrix} \text{learning-} \\ \text{rate parameter} \\ \eta \end{pmatrix} \times \begin{pmatrix} \text{local} \\ \text{gradient} \\ \delta_j(n) \end{pmatrix} \times \begin{pmatrix} \text{input signal} \\ \text{of neuron } j, \\ y_i(n) \end{pmatrix} \quad (4.27)$$

Second, the local gradient $\delta_j(n)$ depends on whether neuron j is an output node or a hidden node:

1. If neuron j is an output node, $\delta_j(n)$ equals the product of the derivative $\phi'_j(v_j(n))$ and the error signal $e_j(n)$, both of which are associated with neuron j ; see Eq. (4.16).
2. If neuron j is a hidden node, $\delta_j(n)$ equals the product of the associated derivative $\phi'_j(v_j(n))$ and the weighted sum of the δ s computed for the neurons in the next hidden or output layer that are connected to neuron j ; see Eq. (4.26).

The Two Passes of Computation

In the application of the back-propagation algorithm, two different passes of computation are distinguished. The first pass is referred to as the forward pass, and the second is referred to as the backward pass.

In the *forward pass*, the synaptic weights remain unaltered throughout the network, and the function signals of the network are computed on a neuron-by-neuron basis. The function signal appearing at the output of neuron j is computed as

$$y_j(n) = \phi(v_j(n)) \quad (4.28)$$

where $v_j(n)$ is the induced local field of neuron j , defined by

$$v_j(n) = \sum_{i=0}^m w_{ji}(n) y_i(n) \quad (4.29)$$

where m is the total number of inputs (excluding the bias) applied to neuron j ; $w_{ji}(n)$ is the synaptic weight connecting neuron i to neuron j ; and $y_i(n)$ is an input signal of neuron j , or, equivalently, the function signal appearing at the output of neuron i . If neuron j is in the first hidden layer of the network, then $m = m_0$ and the index i refers to the i th input terminal of the network, for which we write

$$y_i(n) = x_i(n) \quad (4.30)$$

where $x_i(n)$ is the i th element of the input vector (pattern). If, on the other hand, neuron j is in the output layer of the network, then $m = m_L$ and the index j refers to the j th output terminal of the network, for which we write

$$y_j(n) = o_j(n) \quad (4.31)$$

where $o_j(n)$ is the j th element of the output vector of the multilayer perceptron. This output is compared with the desired response $d_j(n)$, obtaining the error signal $e_j(n)$ for the j th output neuron. Thus, the forward phase of computation begins at the first hidden layer by presenting it with the input vector and terminates at the output layer by computing the error signal for each neuron of this layer.

The backward pass, on the other hand, starts at the output layer by passing the error signals leftward through the network, layer by layer, and recursively computing the δ (i.e., the local gradient) for each neuron. This recursive process permits the synaptic weights of the network to undergo changes in accordance with the delta rule of Eq. (4.27). For a neuron located in the output layer, the δ is simply equal to the error signal of that neuron multiplied by the first derivative of its nonlinearity. Hence, we use Eq. (4.27) to compute the changes to the weights of all the connections feeding into the output layer. Given the δ s for the neurons of the output layer, we next use Eq. (4.26) to compute the δ s for all the neurons in the penultimate layer and therefore the changes to the weights of all connections feeding into it. The recursive computation is continued, layer by layer, by propagating the changes to all synaptic weights in the network.

Note that for the presentation of each training example, the input pattern is fixed—that is, “clamped” throughout the round-trip process, which encompasses the forward pass followed by the backward pass.

Activation Function

The computation of the δ for each neuron of the multilayer perceptron requires knowledge of the derivative of the activation function $\varphi(\cdot)$ associated with that neuron. For this derivative to exist, we require the function $\varphi(\cdot)$ to be continuous. In basic terms, *differentiability* is the only requirement that an activation function has to satisfy. An example of a continuously differentiable nonlinear activation function commonly used in multilayer perceptrons is *sigmoidal nonlinearity*,¹ two forms of which are described here:

1. Logistic Function. This form of sigmoidal nonlinearity, in its general form, is defined by

$$\varphi_j(v_j(n)) = \frac{1}{1 + \exp(-av_j(n))}, \quad a > 0 \quad (4.32)$$

where $v_j(n)$ is the induced local field of neuron j and a is an adjustable positive parameter. According to this nonlinearity, the amplitude of the output lies inside the range $0 \leq y_j \leq 1$. Differentiating Eq. (4.32) with respect to $v_j(n)$, we get

$$\varphi'_j(v_j(n)) = \frac{a \exp(-av_j(n))}{[1 + \exp(-av_j(n))]^2} \quad (4.33)$$

With $y_j(n) = \varphi_j(v_j(n))$, we may eliminate the exponential term $\exp(-av_j(n))$ from Eq. (4.33) and consequently express the derivative $\varphi'_j(v_j(n))$ as

$$\varphi'_j(v_j(n)) = ay_j(n)[1 - y_j(n)] \quad (4.34)$$

For a neuron j located in the output layer, $y_j(n) = o_j(n)$. Hence, we may express the local gradient for neuron j as

$$\begin{aligned} \delta_j(n) &= e_j(n)\varphi'_j(v_j(n)) \\ &= a[d_j(n) - o_j(n)]o_j(n)[1 - o_j(n)], \quad \text{neuron } j \text{ is an output node} \end{aligned} \quad (4.35)$$

where $o_j(n)$ is the function signal at the output of neuron j , and $d_j(n)$ is the desired response for it. On the other hand, for an arbitrary hidden neuron j , we may express the local gradient as

$$\begin{aligned} \delta_j(n) &= \varphi'_j(v_j(n)) \sum_k \delta_k(n)w_{kj}(n) \\ &= ay_j(n)[1 - y_j(n)] \sum_k \delta_k(n)w_{kj}(n), \quad \text{neuron } j \text{ is hidden} \end{aligned} \quad (4.36)$$

Note from Eq. (4.34) that the derivative $\varphi'_j(v_j(n))$ attains its maximum value at $y_j(n) = 0.5$ and its minimum value (zero) at $y_j(n) = 0$, or $y_j(n) = 1.0$. Since the amount of change in a synaptic weight of the network is proportional to the derivative $\varphi'_j(v_j(n))$, it follows that for a sigmoid activation function, the synaptic weights are changed the most for those neurons in the network where the function signals are in their midrange. According to Rumelhart et al. (1986a), it is this feature of back-propagation learning that contributes to its stability as a learning algorithm.

2. Hyperbolic tangent function. Another commonly used form of sigmoidal nonlinearity is the hyperbolic tangent function, which, in its most general form, is defined by

$$\varphi_j(v_j(n)) = a \tanh(bv_j(n)) \quad (4.37)$$

where a and b are positive constants. In reality, the hyperbolic tangent function is just the logistic function rescaled and biased. Its derivative with respect to $v_j(n)$ is given by

$$\begin{aligned} \varphi'_j(v_j(n)) &= ab \operatorname{sech}^2(bv_j(n)) \\ &= ab(1 - \tanh^2(bv_j(n))) \\ &= \frac{b}{a} [a - y_j(n)][a + y_j(n)] \end{aligned} \quad (4.38)$$

For a neuron j located in the output layer, the local gradient is

$$\begin{aligned}\delta_j(n) &= e_j(n)\varphi'_j(v_j(n)) \\ &= \frac{b}{a} [d_j(n) - o_j(n)][a - o_j(n)][a + o_j(n)]\end{aligned}\quad (4.39)$$

For a neuron j in a hidden layer, we have

$$\begin{aligned}\delta_j(n) &= \varphi'_j(v_j(n)) \sum_k \delta_k(n) w_{kj}(n) \\ &= \frac{b}{a} [a - y_j(n)][a + y_j(n)] \sum_k \delta_k(n) w_{kj}(n), \quad \text{neuron } j \text{ is hidden}\end{aligned}\quad (4.40)$$

By using Eqs. (4.35) and (4.36) for the logistic function and Eqs. (4.39) and (4.40) for the hyperbolic tangent function, we may calculate the local gradient δ_j without requiring explicit knowledge of the activation function.

Rate of Learning

The back-propagation algorithm provides an “approximation” to the trajectory in weight space computed by the method of steepest descent. The smaller we make the learning-rate parameter η , the smaller the changes to the synaptic weights in the network will be from one iteration to the next, and the smoother will be the trajectory in weight space. This improvement, however, is attained at the cost of a slower rate of learning. If, on the other hand, we make the learning-rate parameter η too large in order to speed up the rate of learning, the resulting large changes in the synaptic weights assume such a form that the network may become unstable (i.e., oscillatory). A simple method of increasing the rate of learning while avoiding the danger of instability is to modify the delta rule of Eq. (4.15) by including a momentum term, as shown by

$$\Delta w_{ji}(n) = \alpha \Delta w_{ji}(n-1) + \eta \delta_j(n) y_i(n) \quad (4.41)$$

where α is usually a positive number called the *momentum constant*. It controls the feed-back loop acting around $\Delta w_{ji}(n)$, as illustrated in Fig. 4.6, where z^{-1} is the unit-time delay operator. Equation (4.41) is called the *generalized delta rule*²; it includes the delta rule of Eq. (4.15) as a special case (i.e., $\alpha = 0$).

In order to see the effect of the sequence of pattern presentations on the synaptic weights due to the momentum constant α , we rewrite Eq. (4.41) as a time series with index t . The index t goes from the initial time 0 to the current time n . Equation (4.41)

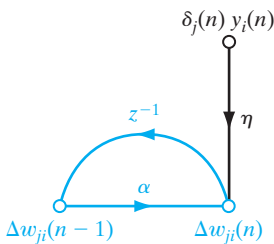


FIGURE 4.6 Signal-flow graph illustrating the effect of momentum constant α , which lies inside the feedback loop.

may be viewed as a first-order difference equation in the weight correction $\Delta w_{ji}(n)$. Solving this equation for $\Delta w_{ji}(n)$, we have

$$\Delta w_{ji}(n) = \eta \sum_{t=0}^n \alpha^{n-t} \delta_j(t) y_i(t) \quad (4.42)$$

which represents a time series of length $n + 1$. From Eqs. (4.13) and (4.16), we note that the product $\delta_j(n) y_i(n)$ is equal to $-\partial \mathcal{E}(n) / \partial w_{ji}(n)$. Accordingly, we may rewrite Eq. (4.42) in the equivalent form

$$\Delta w_{ji}(n) = -\eta \sum_{t=0}^n \alpha^{n-t} \frac{\partial \mathcal{E}(t)}{\partial w_{ji}(t)} \quad (4.43)$$

Based on this relation, we may make the following insightful observations:

1. The current adjustment $\Delta w_{ji}(n)$ represents the sum of an exponentially weighted time series. For the time series to be *convergent*, the momentum constant must be restricted to the range $0 \leq |\alpha| < 1$. When α is zero, the back-propagation algorithm operates without momentum. Also, the momentum constant α can be positive or negative, although it is unlikely that a negative α would be used in practice.
2. When the partial derivative $\partial \mathcal{E}(t) / \partial w_{ji}(t)$ has the same algebraic sign on consecutive iterations, the exponentially weighted sum $\Delta w_{ji}(n)$ grows in magnitude, and consequently the weight $w_{ji}(n)$ is adjusted by a large amount. The inclusion of momentum in the back-propagation algorithm tends to *accelerate descent* in steady downhill directions.
3. When the partial derivative $\partial \mathcal{E}(t) / \partial w_{ji}(t)$ has opposite signs on consecutive iterations, the exponentially weighted sum $\Delta w_{ji}(n)$ shrinks in magnitude, and consequently the weight $w_{ji}(n)$ is adjusted by a small amount. The inclusion of momentum in the back-propagation algorithm has a *stabilizing effect* in directions that oscillate in sign.

The incorporation of momentum in the back-propagation algorithm represents a minor modification to the weight update; however, it may have some beneficial effects on the learning behavior of the algorithm. The momentum term may also have the benefit of preventing the learning process from terminating in a shallow local minimum on the error surface.

In deriving the back-propagation algorithm, it was assumed that the learning-rate parameter is a constant denoted by η . In reality, however, it should be defined as η_{ji} ; that is, the learning-rate parameter should be *connection dependent*. Indeed, many interesting things can be done by making the learning-rate parameter different for different parts of the network. We provide more detail on this issue in subsequent sections.

It is also noteworthy that in the application of the back-propagation algorithm, we may choose all the synaptic weights in the network to be adjustable, or we may constrain any number of weights in the network to remain fixed during the adaptation process. In the latter case, the error signals are back propagated through the network in the usual manner; however, the fixed synaptic weights are left unaltered. This can be done simply by making the learning-rate parameter η_{ji} for synaptic weight w_{ji} equal to zero.

Stopping Criteria

In general, the back-propagation algorithm cannot be shown to converge, and there are no well-defined criteria for stopping its operation. Rather, there are some reasonable criteria, each with its own practical merit, that may be used to terminate the weight adjustments. To formulate such a criterion, it is logical to think in terms of the unique properties of a *local* or *global minimum* of the error surface.³ Let the weight vector $\mathbf{w}^*$ denote a minimum, be it local or global. A necessary condition for $\mathbf{w}^*$ to be a minimum is that the gradient vector $\mathbf{g}(\mathbf{w})$ (i.e., first-order partial derivative) of the error surface with respect to the weight vector $\mathbf{w}$ must be zero at $\mathbf{w} = \mathbf{w}^*$. Accordingly, we may formulate a sensible convergence criterion for back-propagation learning as follows (Kramer and Sangiovanni-Vincentelli, 1989):

The back-propagation algorithm is considered to have converged when the Euclidean norm of the gradient vector reaches a sufficiently small gradient threshold.

The drawback of this convergence criterion is that, for successful trials, learning times may be long. Also, it requires the computation of the gradient vector $\mathbf{g}(\mathbf{w})$.

Another unique property of a minimum that we can use is the fact that the cost function $\mathcal{E}_{\text{av}}(\mathbf{w})$ is stationary at the point $\mathbf{w} = \mathbf{w}^*$. We may therefore suggest a different criterion of convergence:

The back-propagation algorithm is considered to have converged when the absolute rate of change in the average squared error per epoch is sufficiently small.

The rate of change in the average squared error is typically considered to be small enough if it lies in the range of 0.1 to 1 percent per epoch. Sometimes a value as small as 0.01 percent per epoch is used. Unfortunately, this criterion may result in a premature termination of the learning process.

There is another useful, and theoretically supported, criterion for convergence: After each learning iteration, the network is tested for its generalization performance. The learning process is stopped when the generalization performance is adequate or when it is apparent that the generalization performance has peaked; see Section 4.13 for more details.

Summary of the Back-Propagation Algorithm

Figure 4.1 presents the architectural layout of a multilayer perceptron. The corresponding signal-flow graph for back-propagation learning, incorporating both the forward and backward phases of the computations involved in the learning process, is presented in Fig. 4.7 for the case of $L = 2$ and $m_0 = m_1 = m_2 = 3$. The top part of the signal-flow graph accounts for the forward pass. The lower part of the signal-flow graph accounts for the backward pass, which is referred to as a *sensitivity graph* for computing the local gradients in the back-propagation algorithm (Narendra and Parthasarathy, 1990).

Earlier, we mentioned that the sequential updating of weights is the preferred method for on-line implementation of the back-propagation algorithm. For this mode

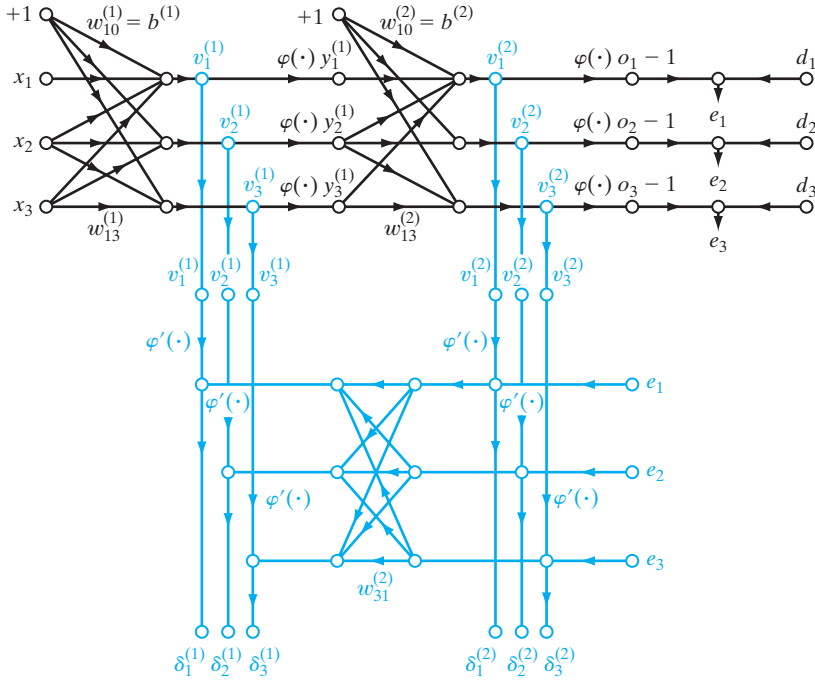


FIGURE 4.7 Signal-flow graphical summary of back-propagation learning. Top part of the graph: forward pass. Bottom part of the graph: backward pass.

of operation, the algorithm cycles through the training sample $\{(\mathbf{x}(n), \mathbf{d}(n))\}_{n=1}^N$ as follows:

- 1. Initialization.** Assuming that no prior information is available, pick the synaptic weights and thresholds from a uniform distribution whose mean is zero and whose variance is chosen to make the standard deviation of the induced local fields of the neurons lie at the transition between the linear and standards parts of the sigmoid activation function.

- 2. Presentations of Training Examples.** Present the network an epoch of training examples. For each example in the sample, ordered in some fashion, perform the sequence of forward and backward computations described under points 3 and 4, respectively.

- 3. Forward Computation.** Let a training example in the epoch be denoted by $(\mathbf{x}(n), \mathbf{d}(n))$, with the input vector $\mathbf{x}(n)$ applied to the input layer of sensory nodes and the desired response vector $\mathbf{d}(n)$ presented to the output layer of computation nodes. Compute the induced local fields and function signals of the network by proceeding forward through the network, layer by layer. The induced local field $v_j^{(l)}(n)$ for neuron j in layer l is

$$v_j^{(l)}(n) = \sum_i w_{ji}^{(l)}(n) y_i^{(l-1)}(n) \quad (4.44)$$

where $y_i^{(l-1)}(n)$ is the output (function) signal of neuron i in the previous layer $l-1$ at iteration n , and $w_{ji}^{(l)}(n)$ is the synaptic weight of neuron j in layer l that is fed from neuron i in layer $l-1$. For $i=0$, we have $y_0^{(l-1)}(n) = +1$, and $w_{j0}^{(l)}(n) = b_j^{(l)}(n)$ is the bias applied to neuron j in layer l . Assuming the use of a sigmoid function, the output signal of neuron j in layer l is

$$y_j^{(l)} = \varphi_j(v_j(n))$$

If neuron j is in the first hidden layer (i.e., $l=1$), set

$$y_j^{(0)}(n) = x_j(n)$$

where $x_j(n)$ is the j th element of the input vector $\mathbf{x}(n)$. If neuron j is in the output layer (i.e., $l=L$, where L is referred to as the *depth* of the network), set

$$y_j^{(L)} = o_j(n)$$

Compute the error signal

$$e_j(n) = d_j(n) - o_j(n) \quad (4.45)$$

where $d_j(n)$ is the j th element of the desired response vector $\mathbf{d}(n)$.

4. Backward Computation. Compute the δ s (i.e., local gradients) of the network, defined by

$$\delta_j^{(l)}(n) = \begin{cases} e_j^{(L)}(n)\varphi_j'(v_j^{(L)}(n)) & \text{for neuron } j \text{ in output layer } L \\ \varphi_j'(v_j^{(l)}(n)) \sum_k \delta_k^{(l+1)}(n)w_{kj}^{(l+1)}(n) & \text{for neuron } j \text{ in hidden layer } l \end{cases} \quad (4.46)$$

where the prime in $\varphi_j'(\cdot)$ denotes differentiation with respect to the argument. Adjust the synaptic weights of the network in layer l according to the generalized delta rule

$$w_{ji}^{(l)}(n+1) = w_{ji}^{(l)}(n) + \alpha[\Delta w_{ji}^{(l)}(n-1)] + \eta\delta_j^{(l)}(n)y_i^{(l-1)}(n) \quad (4.47)$$

where η is the learning-rate parameter and α is the momentum constant.

5. Iteration. Iterate the forward and backward computations under points 3 and 4 by presenting new epochs of training examples to the network until the chosen stopping criterion is met.

Notes: The order of presentation of training examples should be randomized from epoch to epoch. The momentum and learning-rate parameter are typically adjusted (and usually decreased) as the number of training iterations increases. Justification for these points will be presented later.

4.5 XOR PROBLEM

In Rosenblatt's single-layer perceptron, there are no hidden neurons. Consequently, it cannot classify input patterns that are not linearly separable. However, nonlinearly separable patterns commonly occur. For example, this situation arises in the *exclusive-OR (XOR) problem*, which may be viewed as a special case of a more general problem, namely, that of classifying points in the *unit hypercube*. Each point in the hypercube is in either class 0 or class 1. However, in the special case of the XOR problem, we need

consider only the four corners of a *unit square* that correspond to the input patterns (0,0), (0,1), (1,1), and (1,0), where a single bit (i.e., binary digit) changes as we move from one corner to the next. The first and third input patterns are in class 0, as shown by

$$0 \oplus 0 = 0$$

and

$$1 \oplus 1 = 0$$

where $\oplus$ denotes the exclusive-OR Boolean function operator. The input patterns (0,0) and (1,1) are at opposite corners of the unit square, yet they produce the identical output 0. On the other hand, the input patterns (0,1) and (1,0) are also at opposite corners of the square, but they are in class 1, as shown by

$$0 \oplus 1 = 1$$

and

$$1 \oplus 0 = 1$$

We first recognize that the use of a single neuron with two inputs results in a straight line for a decision boundary in the input space. For all points on one side of this line, the neuron outputs 1; for all points on the other side of the line, it outputs 0. The position and orientation of the line in the input space are determined by the synaptic weights of the neuron connected to the input nodes and the bias applied to the neuron. With the input patterns (0,0) and (1,1) located on opposite corners of the unit square, and likewise for the other two input patterns (0,1) and (1,0), it is clear that we cannot construct a straight line for a decision boundary so that (0,0) and (0,1) lie in one decision region and (0,1) and (1,0) lie in the other decision region. In other words, the single-layer perceptron cannot solve the XOR problem.

However, we may solve the XOR problem by using a single hidden layer with two neurons, as in Fig. 4.8a (Touretzky and Pomerleau, 1989). The signal-flow graph of the network is shown in Fig. 4.8b. The following assumptions are made here:

- Each neuron is represented by a McCulloch–Pitts model, which uses a threshold function for its activation function.
- Bits 0 and 1 are represented by the levels 0 and +1, respectively.

The top neuron, labeled as “Neuron 1” in the hidden layer, is characterized as

$$\begin{aligned} w_{11} &= w_{12} = +1 \\ b_1 &= -\frac{3}{2} \end{aligned}$$

The slope of the decision boundary constructed by this hidden neuron is equal to -1 and positioned as in Fig. 4.9a. The bottom neuron, labeled as “Neuron 2” in the hidden layer, is characterized as

$$\begin{aligned} w_{21} &= w_{22} = +1 \\ b_2 &= -\frac{1}{2} \end{aligned}$$

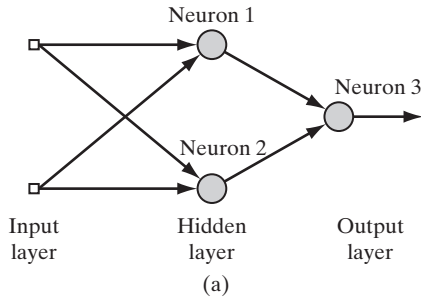
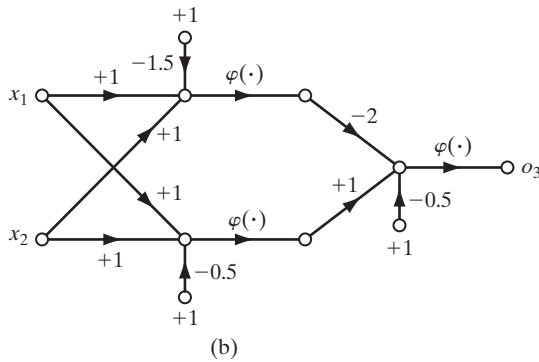


FIGURE 4.8 (a) Architectural graph of network for solving the XOR problem. (b) Signal-flow graph of the network.



The orientation and position of the decision boundary constructed by this second hidden neuron are as shown in Fig. 4.9b.

The output neuron, labeled as “Neuron 3” in Fig. 4.8a, is characterized as

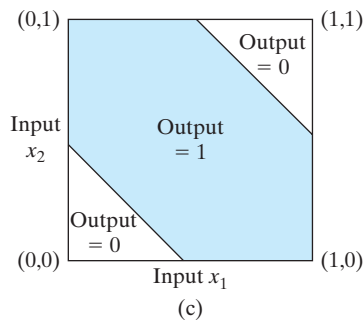
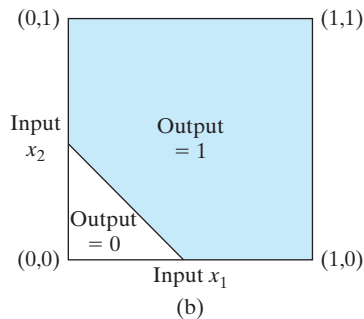
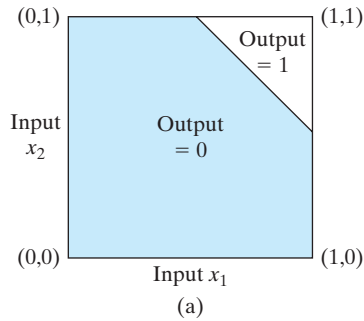
$$w_{31} = -2$$

$$w_{32} = +1$$

$$b_3 = -\frac{1}{2}$$

The function of the output neuron is to construct a linear combination of the decision boundaries formed by the two hidden neurons. The result of this computation is shown in Fig. 4.9c. The bottom hidden neuron has an excitatory (positive) connection to the output neuron, whereas the top hidden neuron has an inhibitory (negative) connection to the output neuron. When both hidden neurons are off, which occurs when the input pattern is (0,0), the output neuron remains off. When both hidden neurons are on, which occurs when the input pattern is (1,1), the output neuron is switched off again because the inhibitory effect of the larger negative weight connected to the top hidden neuron overpowers the excitatory effect of the positive weight connected to the bottom hidden neuron. When the top hidden neuron is off and the bottom hidden neuron is on, which occurs when the input pattern is (0,1) or (1,0), the output neuron is switched on because of the excitatory effect of the positive weight connected to the bottom hidden neuron. Thus, the network of Fig. 4.8a does indeed solve the XOR problem.

FIGURE 4.9 (a) Decision boundary constructed by hidden neuron 1 of the network in Fig. 4.8. (b) Decision boundary constructed by hidden neuron 2 of the network. (c) Decision boundaries constructed by the complete network.



4.6 HEURISTICS FOR MAKING THE BACK-PROPAGATION ALGORITHM PERFORM BETTER

It is often said that the design of a neural network using the back-propagation algorithm is more of an art than a science, in the sense that many of the factors involved in the design are the results of one's own personal experience. There is some truth in this statement. Nevertheless, there are methods that will significantly improve the back-propagation algorithm's performance, as described here:

1. *Stochastic versus batch update.* As mentioned previously, the stochastic (sequential) mode of back-propagation learning (involving pattern-by-pattern updating) is computationally faster than the batch mode. This is especially true when the

training data sample is large and highly redundant. (Highly redundant data pose computational problems for the estimation of the Jacobian required for the batch update.)

2. Maximizing information content. As a general rule, every training example presented to the back-propagation algorithm should be chosen on the basis that its information content is the largest possible for the task at hand (LeCun, 1993). Two ways of realizing this choice are as follows:

- Use an example that results in the largest training error.
- Use an example that is radically different from all those previously used.

These two heuristics are motivated by a desire to search more of the weight space.

In pattern-classification tasks using sequential back-propagation learning, a simple and commonly used technique is to randomize (i.e., shuffle) the order in which the examples are presented to the multilayer perceptron from one epoch to the next. Ideally, the randomization ensure that successive examples in an epoch presented to the network rarely belong to the same class.

3. Activation function. Insofar as the speed of learning is concerned, the preferred choice is to use a sigmoid activation function that is an *odd function of its argument*, as shown by

$$\varphi(-v) = -\varphi(v)$$

This condition is satisfied by the hyperbolic function

$$\varphi(v) = a \tanh(bv)$$

as shown in Fig. 4.10, but not the logistic function. Suitable values for the constraints a and b in the formula for $\varphi(v)$ are as follows (LeCun, 1993):

$$a = 1.7159$$

and

$$b = \frac{2}{3}$$

The hyperbolic tangent function $\varphi(v)$ of Fig. 4.10 has the following useful properties:

- $\varphi(1) = 1$ and $\varphi(-1) = -1$.
- At the origin, the slope (i.e., effective gain) of the activation function is close to unity, as shown by

$$\begin{aligned} \varphi(0) &= ab \\ &= 1.7159 \left(\frac{2}{3} \right) \\ &= 1.1424 \end{aligned}$$

- The second derivative of $\varphi(v)$ attains its maximum value at $v = 1$.

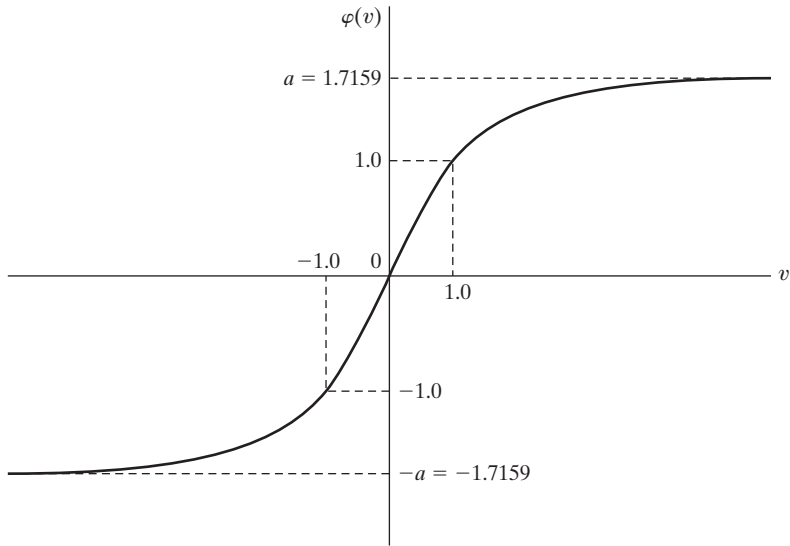


FIGURE 4.10 Graph of the hyperbolic tangent function $\varphi(v) = a \tanh(bv)$ for $a = 1.7159$ and $b = 2/3$. The recommended target values are $+1$ and -1 .

4. Target values. It is important that the target values (desired response) be chosen within the range of the sigmoid activation function. More specifically, the desired response d_j for neuron j in the output layer of the multilayer perceptron should be *offset* by some amount ε away from the limiting value of the sigmoid activation function, depending on whether the limiting value is positive or negative. Otherwise, the back-propagation algorithm tends to drive the free parameters of the network to infinity and thereby slow down the learning process by driving the hidden neurons into saturation. To be specific, consider the hyperbolic tangent function of Fig. 4.10. For the limiting value $+a$, we set

$$d_j = a - \varepsilon$$

and for the limiting value of $-a$, we set

$$d_j = -a + \varepsilon$$

where ε is an appropriate positive constant. For the choice of $a = 1.7159$ used in Fig. 4.10, we may set $\varepsilon = 0.7159$, in which case the target value (desired response) d_j can be conveniently chosen as ± 1 , as indicated in the figure.

5. Normalizing the inputs. Each input variable should be *preprocessed* so that its mean value, averaged over the entire training sample, is close to zero, or else it will be small compared to its standard deviation (LeCun, 1993). To appreciate the practical significance of this rule, consider the extreme case where the input variables are consistently positive. In this situation, the synaptic weights of a neuron in the first hidden layer can only increase together or decrease together. Accordingly, if the weight vector of that

neuron is to change direction, it can do so only by zigzagging its way through the error surface, which is typically slow and should therefore be avoided.

In order to accelerate the back-propagation learning process, the normalization of the inputs should also include two other measures (LeCun, 1993):

- The input variables contained in the training set should be *uncorrelated*; this can be done by using principal-components analysis, to be discussed in Chapter 8.
- The decorrelated input variables should be scaled so that their *covariances are approximately equal*, thereby ensuring that the different synaptic weights in the network learn at approximately the same speed.

Figure 4.11 illustrates the results of three normalization steps: mean removal, decorrelation, and covariance equalization, applied in that order.

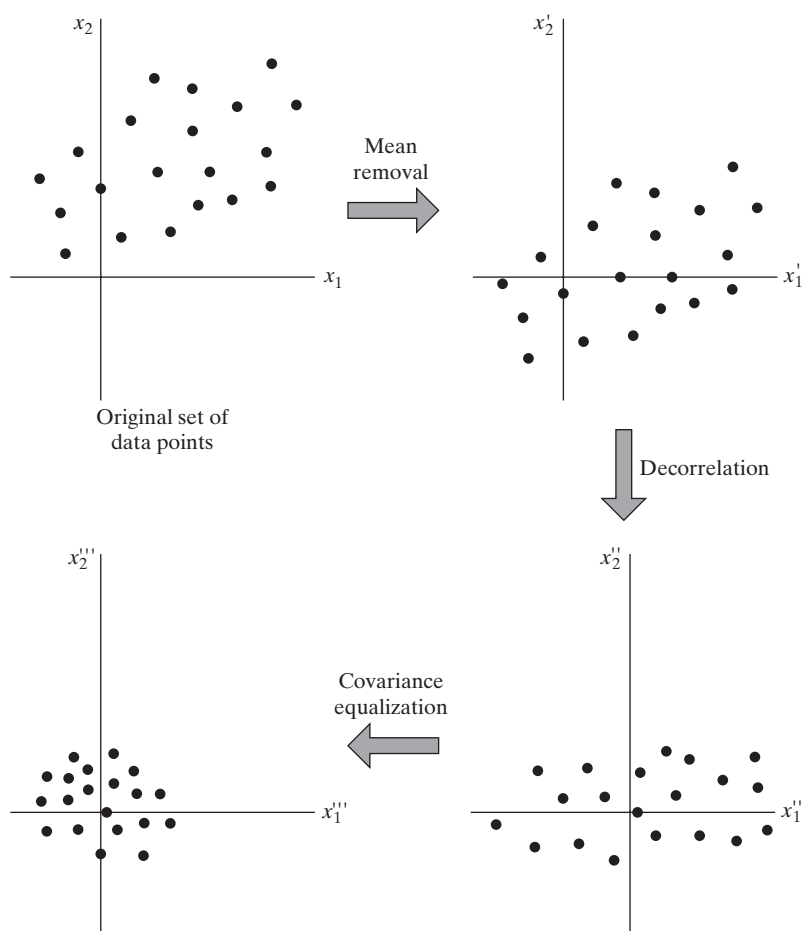


FIGURE 4.11 Illustrating the operation of mean removal, decorrelation, and covariance equalization for a two-dimensional input space.

It is also of interest to note that when the inputs are transformed in the manner illustrated in Fig. 4.11 and used in conjunction with the hyperbolic tangent function specified in Fig. 4.10, the variance of the individual neural outputs in the multilayer perceptron will be close to unity (Orr and Müller, 1998). The rationale for this statement is that the effective gain of the sigmoid function over its useful range is roughly unity.

6. Initialization. A good choice for the initial values of the synaptic weights and thresholds of the network can be of tremendous help in a successful network design. The key question is: What is a good choice?

When the synaptic weights are assigned large initial values, it is highly likely that the neurons in the network will be driven into saturation. If this happens, the local gradients in the back-propagation algorithm assume small values, which in turn will cause the learning process to slow down. However, if the synaptic weights are assigned small initial values, the back-propagation algorithm may operate on a very flat area around the origin of the error surface; this is particularly true in the case of sigmoid functions such as the hyperbolic tangent function. Unfortunately, the origin is a *saddle point*, which refers to a stationary point where the curvature of the error surface across the saddle is negative and the curvature along the saddle is positive. For these reasons, the use of both large and small values for initializing the synaptic weights should be avoided. The proper choice of initialization lies somewhere between these two extreme cases.

To be specific, consider a multilayer perceptron using the hyperbolic tangent function for its activation functions. Let the bias applied to each neuron in the network be set to zero. We may then express the induced local field of neuron j as

$$v_j = \sum_{i=1}^m w_{ji} y_i$$

Let it be assumed that the inputs applied to each neuron in the network have zero mean and unit variance, as shown by

$$\mu_y = \mathbb{E}[y_i] = 0 \quad \text{for all } i$$

and

$$\sigma_y^2 = \mathbb{E}[(y_i - \mu_i)^2] = \mathbb{E}[y_i^2] = 1 \quad \text{for all } i$$

Let it be further assumed that the inputs are uncorrelated, as shown by

$$\mathbb{E}[y_i y_k] = \begin{cases} 1 & \text{for } k = i \\ 0 & \text{for } k \neq i \end{cases}$$

and that the synaptic weights are drawn from a uniformly distributed set of numbers with zero mean, that is,

$$\mu_w = \mathbb{E}[w_{ji}] = 0 \quad \text{for all } (j, i) \text{ pairs}$$

and variance

$$\sigma_w^2 = \mathbb{E}[(w_{ji} - \mu_w)^2] = \mathbb{E}[w_{ji}^2] \quad \text{for all } (j, i) \text{ pairs}$$

Accordingly, we may express the mean and variance of the induced local field v_j as

$$\mu_v = \mathbb{E}[v_j] = \mathbb{E}\left[\sum_{i=1}^m w_{ji}y_i\right] = \sum_{i=1}^m \mathbb{E}[w_{ji}]\mathbb{E}[y_i] = 0$$

and

$$\begin{aligned}\sigma_v^2 &= \mathbb{E}[(v_j - \mu_v)^2] = \mathbb{E}[v_j^2] \\ &= \mathbb{E}\left[\sum_{i=1}^m \sum_{k=1}^m w_{ji}w_{jk}y_iy_k\right] \\ &= \sum_{i=1}^m \sum_{k=1}^m \mathbb{E}[w_{ji}w_{jk}]\mathbb{E}[y_iy_k] \\ &= \sum_{i=1}^m \mathbb{E}[w_{ji}^2] \\ &= m\sigma_w^2\end{aligned}$$

where m is the number of synaptic connections of a neuron.

In light of this result, we may now describe a good strategy for initializing the synaptic weights so that the standard deviation of the induced local field of a neuron lies in the transition area between the linear and saturated parts of its sigmoid activation function. For example, for the case of a hyperbolic tangent function with parameters a and b used in Fig. 4.10, this objective is satisfied by setting $\sigma_v = 1$ in the previous equation, in which case we obtain the following (LeCun, 1993):

$$\sigma_w = m^{-1/2} \quad (4.48)$$

Thus, it is desirable for the uniform distribution, from which the synaptic weights are selected, to have a mean of zero and a variance equal to the reciprocal of the number of synaptic connections of a neuron.

7. Learning from hints. Learning from a sample of training examples deals with an unknown input–output mapping function $f(\cdot)$. In effect, the learning process exploits the information contained in the examples about the function $f(\cdot)$ to *infer* an approximate implementation of it. The process of learning from examples may be generalized to *include learning from hints*, which is achieved by allowing prior information that we may have about the function $f(\cdot)$ to be included in the learning process (Abu-Mostafa, 1995). Such information may include invariance properties, symmetries, or any other knowledge about the function $f(\cdot)$ that may be used to accelerate the search for its approximate realization and, more importantly, to improve the quality of the final estimate. The use of Eq. (4.48) is an example of how this is achieved.

8. Learning rates. All neurons in the multilayer perceptron should ideally learn at the same rate. The last layers usually have larger local gradients than the layers at the front end of the network. Hence, the learning-rate parameter η should be assigned a

smaller value in the last layers than in the front layers of the multilayer perceptron. Neurons with many inputs should have a smaller learning-rate parameter than neurons with few inputs so as to maintain a similar learning time for all neurons in the network. In LeCun (1993), it is suggested that for a given neuron, the learning rate should be inversely proportional to the square root of synaptic connections made to that neuron.

4.7 COMPUTER EXPERIMENT: PATTERN CLASSIFICATION

In this computer experiment, we resume the sequence of pattern-classification experiments performed first in Chapter 1 using Rosenblatt's perceptron and then in Chapter 2 using the method of least squares. For both experiments, we used training and test data generated by randomly sampling the *double-moon* structure pictured in Fig. 1.8. In each of the experiments, we considered two cases, one employing linearly separable patterns and the other employing nonlinearly separable patterns. The perceptron worked perfectly fine for the linearly separable setting of $d = 1$, but the method of least squares required a larger separation between the two moons for perfect classification. In any event, they both failed the nonlinearly separable setting of $d = -4$.

The objective of the computer experiment presented herein is twofold:

1. to demonstrate that the multilayer perceptron, trained with the back-propagation algorithm, is capable of classifying nonlinearly separable test data;
2. to find a more difficult case of nonlinearly separable patterns for which the multilayer perceptron fails the double-moon classification test.

The specifications of the multilayer perceptron used in the experiment are as follows:

Size of the input layer: $m_0 = 2$

Size of the (only) hidden layer: $m_1 = 20$

Size of the output layer: $m_2 = 1$

Activation function: hyperbolic tangent function $\varphi(v) = \frac{1 - \exp(-2v)}{1 + \exp(-2v)}$

Threshold setting: zero

Learning-rate parameter η : annealed linearly from 10^{-1} down to 10^{-5}

The experiment is carried out in two parts, one corresponding to the vertical separation $d = -4$, and the other corresponding to $d = -5$:

(a) Vertical separation $d = -4$.

Figure 4.12 presents the results of the MLP experiment for the length of separation between the two moons of $d = -4$. Part (a) of the figure displays the learning curve resulting from the training session. We see that the learning curve reached convergence effectively in about 15 epochs of training. Part (b) of the figure displays the optimal nonlinear decision boundary computed by the MLP. Most important, perfect classification of the two patterns was achieved, with no classification errors. This perfect performance is attributed to the hidden layer of the MLP.

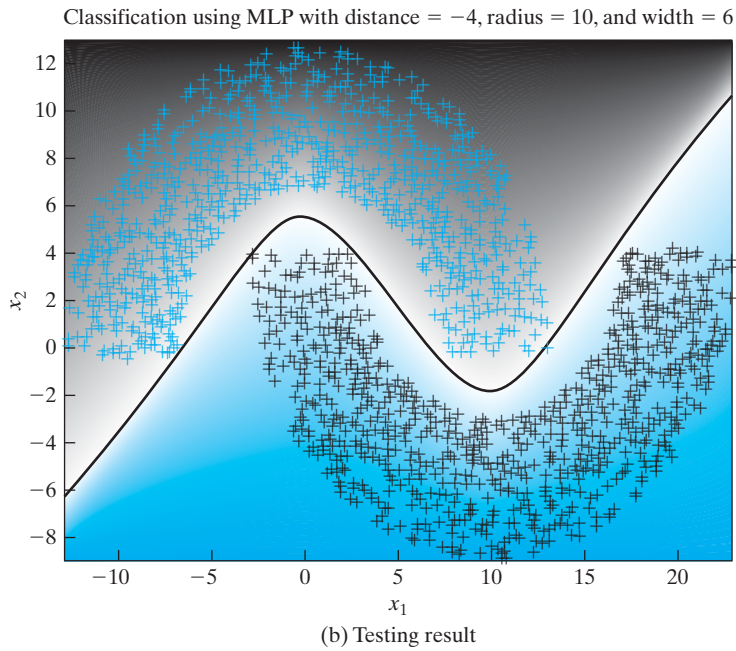
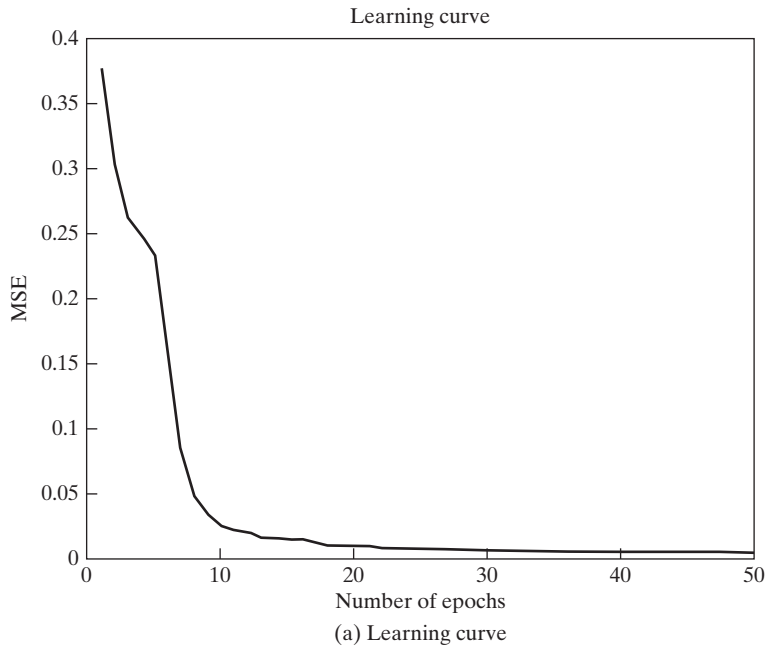


FIGURE 4.12 Results of the computer experiment on the back-propagation algorithm applied to the MLP with distance $d = -4$. MSE stands for mean-square error.

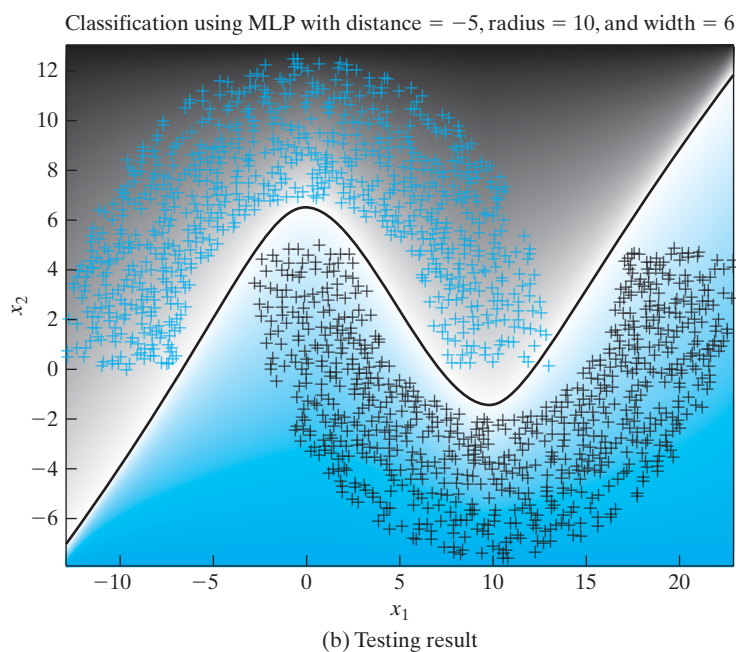
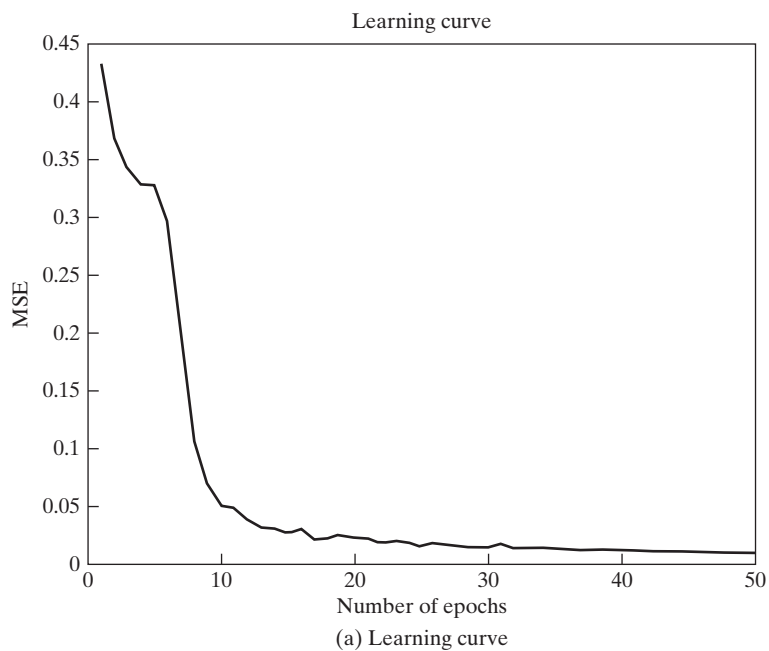


FIGURE 4.13 Results of the computer experiment on the back-propagation algorithm applied to the MLP with distance $d = -5$.

(b) *Vertical separation $d = -5$.*

To challenge the multilayer perceptron with a more difficult pattern-classification task, we reduced the vertical separation between the two moons to $d = -5$. The results of this second part of the experiment are presented in Fig. 4.13. The learning curve of the back-propagation algorithm, plotted in part (a) of the figure, shows a slower rate of convergence, roughly three times that for the easier case of $d = -4$. Moreover, the testing results plotted in part (b) of the figure reveal three classification errors in a testing set of 2,000 data points, representing an error rate of 0.15 percent.

The decision boundary is computed by finding the coordinates x_1 and x_2 pertaining to the input vector $\mathbf{x}$, for which the response of the output neuron is zero on the premise that the two classes of the experiment are equally likely. Accordingly, when a threshold of zero is exceeded, a decision is made in favor of one class; otherwise, the decision is made in favor of the other class. This procedure is followed on all the double-moon classification experiments reported in the book.

4.8 BACK PROPAGATION AND DIFFERENTIATION

Back propagation is a specific technique for implementing *gradient descent* in weight space for a multilayer perceptron. The basic idea is to efficiently compute *partial derivatives* of an approximating function $F(\mathbf{w}, \mathbf{x})$ realized by the network with respect to all the elements of the adjustable weight vector $\mathbf{w}$ for a given value of input vector $\mathbf{x}$. Herein lies the computational power of the back-propagation algorithm.⁴

To be specific, consider a multilayer perceptron with an input layer of m_0 nodes, two hidden layers, and a single output neuron, as depicted in Fig. 4.14. The elements of the weight vector $\mathbf{w}$ are ordered by layer (starting from the first hidden layer), then by neurons in a layer, and then by the number of a synapse within a neuron. Let $w_{ji}^{(l)}$ denote the synaptic weight from neuron i to neuron j in layer $l = 1, 2, \dots$. For $l = 1$, corresponding to the first hidden layer, the index i refers to a source node rather than to a

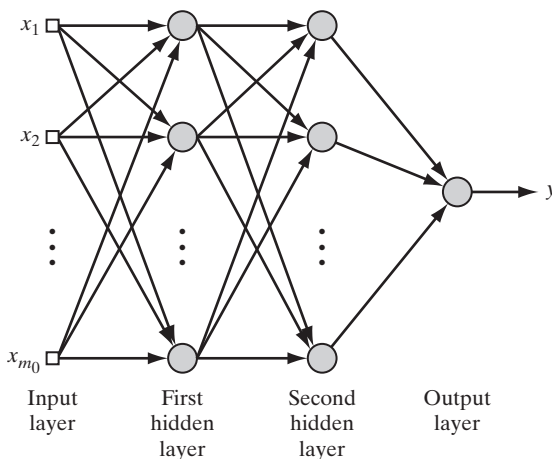


FIGURE 4.14 Multilayer perceptron with two hidden layers and one output neuron.

neuron. For $l = 3$, corresponding to the output layer in Fig. 4.14, we have $j = 1$. We wish to evaluate the derivatives of the function $F(\mathbf{w}, \mathbf{x})$ with respect to all the elements of the weight vector $\mathbf{w}$ for a specified input vector $\mathbf{x} = [x_1, x_2, \dots, x_{m_0}]^T$. We have included the weight vector $\mathbf{w}$ as an argument of the function F in order to focus attention on it. For example, for $l = 2$ (i.e., a single hidden layer and a linear output layer), we have

$$F(\mathbf{w}, \mathbf{x}) = \sum_{j=0}^{m_1} w_{oj} \varphi \left(\sum_{i=0}^{m_0} w_{ji} x_i \right) \quad (4.49)$$

where $\mathbf{w}$ is the ordered weight vector and $\mathbf{x}$ is the input vector.

The multilayer perceptron of Fig. 4.14 is parameterized by an *architecture* $\mathcal{A}$ (representing a discrete parameter) and a *weight vector* $\mathbf{w}$ (made up of continuous elements). Let $\mathcal{A}_j^{(l)}$ denote that part of the architecture extending from the input layer ($l = 0$) to node j in layer $l = 1, 2, 3$. Accordingly, we may write

$$F(\mathbf{w}, \mathbf{x}) = \varphi(\mathcal{A}_1^{(3)}) \quad (4.50)$$

where φ is the activation function. However, $\mathcal{A}_1^{(3)}$ is to be interpreted merely as an architectural symbol rather than a variable. Thus, adapting Eqs. (4.2), (4.4), (4.13), and (4.25) for use in this new situation, we obtain the formulas

$$\frac{\partial F(\mathbf{w}, \mathbf{x})}{\partial w_{1k}^{(3)}} = \varphi'(\mathcal{A}_1^{(3)}) \varphi(\mathcal{A}_k^{(2)}) \quad (4.51)$$

$$\frac{\partial F(\mathbf{w}, \mathbf{x})}{\partial w_{kj}^{(2)}} = \varphi'(\mathcal{A}_1^{(3)}) \varphi'(\mathcal{A}_k^{(2)}) \varphi(\mathcal{A}_j^{(1)}) w_{1k}^{(3)} \quad (4.52)$$

$$\frac{\partial F(\mathbf{w}, \mathbf{x})}{\partial w_{ji}^{(1)}} = \varphi'(\mathcal{A}_1^{(3)}) \varphi'(\mathcal{A}_j^{(1)}) x_i \left[\sum_k w_{1k}^{(3)} \varphi'(\mathcal{A}_k^{(2)}) w_{kj}^{(2)} \right] \quad (4.53)$$

where φ' is the partial derivative of the nonlinearity φ with respect to its argument and x_i is the i th element of the input vector $\mathbf{x}$. In a similar way, we may derive the equations for the partial derivatives of a general network with more hidden layers and more neurons in the output layer.

Equations (4.51) through (4.53) provide the basis for calculating the sensitivity of the network function $F(\mathbf{w}, \mathbf{x})$ with respect to variations in the elements of the weight vector $\mathbf{w}$. Let ω denote an element of the weight vector $\mathbf{w}$. The *sensitivity* of $F(\mathbf{w}, \mathbf{x})$ with respect to ω , is formally defined by

$$S_{\omega}^F = \frac{\partial F / F}{\partial \omega / \omega}$$

It is for this reason that we refer to the lower part of the signal-flow graph in Fig. 4.7 as a “sensitivity graph.”

The Jacobian

Let W denote the total number of free parameters (i.e., synaptic weights and biases) of a multilayer perceptron, which are ordered in a manner described to form the weight vector $\mathbf{w}$. Let N denote the total number of examples used to train the network. Using back

propagation, we may compute a set of W partial derivatives of the approximating function $F[\mathbf{w}, \mathbf{x}(n)]$ with respect to the elements of the weight vector $\mathbf{w}$ for a specific example $\mathbf{x}(n)$ in the training sample. Repeating these computations for $n = 1, 2, \dots, N$, we end up with an N -by- W matrix of partial derivatives. This matrix is called the *Jacobian* $\mathbf{J}$ of the multilayer perceptron evaluated at $\mathbf{x}(n)$. Each row of the Jacobian corresponds to a particular example in the training sample.

There is experimental evidence to suggest that many neural network training problems are intrinsically *ill conditioned*, leading to a Jacobian $\mathbf{J}$ that is almost rank deficient (Saarinen et al., 1991). The *rank* of a matrix is equal to the number of linearly independent columns or rows in the matrix, whichever one is smallest. The Jacobian $\mathbf{J}$ is said to be *rank deficient* if its rank is less than $\min(N, W)$. Any rank deficiency in the Jacobian causes the back-propagation algorithm to obtain only partial information of the possible search directions. Rank deficiency also causes training times to be long.

4.9 THE HESSIAN AND ITS ROLE IN ON-LINE LEARNING

The *Hessian matrix*, or simply the *Hessian*, of the cost function $\mathcal{E}_{av}(\mathbf{w})$, denoted by $\mathbf{H}$, is defined as the second derivative of $\mathcal{E}_{av}(\mathbf{w})$ with respect to the weight vector $\mathbf{w}$, as shown by

$$\mathbf{H} = \frac{\partial^2 \mathcal{E}_{av}(\mathbf{w})}{\partial \mathbf{w}^2} \quad (4.54)$$

The Hessian plays an important role in the study of neural networks; specifically, we mention the following points⁵:

1. The eigenvalues of the Hessian have a profound influence on the dynamics of back-propagation learning.
2. The inverse of the Hessian provides a basis for pruning (i.e., deleting) insignificant synaptic weights from a multilayer perceptron; this issue will be discussed in Section 4.14.
3. The Hessian is basic to the formulation of second-order optimization methods as an alternative to back-propagation learning, to be discussed in Section 4.16.

In this section, we confine our attention to point 1.

In Chapter 3, we indicated that the eigenstructure of the Hessian has a profound influence on the convergence properties of the LMS algorithm. So it is also with the back-propagation algorithm, but in a much more complicated way. Typically, the Hessian of the error surface pertaining to a multilayer perceptron trained with the back-propagation algorithm has the following composition of eigenvalues (LeCun et al., 1998):

- a small number of small eigenvalues,
- a large number of medium-sized eigenvalues, and
- a small number of large eigenvalues.

There is therefore a wide spread in the eigenvalues of the Hessian.